
특이점이 온 개발자 - MSA 아키텍처

모놀리식에서 마이크로서비스로 전환하는 과정에서 배우는 MSA
아키텍처

동기에서 비동기로, 단계별로 바뀌보면 MSA의 구조가 보인다. 하나의 쇼핑몰 프로젝트로 서비스 분리부터 Kafka, WebSocket, Kubernetes까지 MSA 전체 조감도를 훑는다.

최주호, 류재성, 김주혁 지음

특이점이 온 개발자 - MSA 아키텍처

목차

챕터 0. 프롤로그	4
개발 환경 준비	7
챕터 1. MSA란 무엇인가?	9
1.1 모놀리식 : 쇼핑몰을 하나의 서버로 만들면 어떻게 될까?	9
1.2 마이크로서비스 : 역할을 나눈다	11
1.3 시스템 설계 : 우리가 만들 서비스 구조	12
1.4 분산 트랜잭션 : MSA의 핵심 과제	14
1.5 이 책의 학습 흐름	17
정리	18
챕터 2. 동기식 MSA 구현 — 서비스를 연결하다	19
2.1 이야기의 시작 — 네 개의 서비스가 만나다	19
2.2 공통 설정 : 모든 서비스가 공유하는 뼈대	21
2.3 회원 서비스 : JWT로 로그인하다	22
2.4 상품 서비스 : 재고를 관리하다	23
2.5 배달 서비스 : 배달을 생성하고 취소하다	25
2.6 주문 서비스 : 보상 트랜잭션의 현장 (핵심)	28
2.7 Docker Compose : 네 개의 서비스를 한 번에 실행하다	37
정리	44
챕터 3. 클린 아키텍처와 Kubernetes 운영 환경	45
3.1 챕터 2가 남긴 두 가지 숙제	45
3.2 UseCase : 왜 인터페이스인가	46
3.3 UseCase 인터페이스 도입	47
3.4 MySQL : 운영 데이터베이스 연결과 프로파일 분리	51
3.5 Docker : 이미지 빌드와 인프라 구성	52
3.6 Kubernetes : YAML로 선언하는 배포	55
3.7 Minikube : 실행 및 결과 확인	59

정리	63
챕터 4. 비동기 MSA – Kafka로 서비스를 분리하다	65
4.1 한 서비스의 장애가 전체를 멈추다	65
4.2 Kafka : 메시지를 전달하는 우체국	66
4.3 Orchestration Saga : 지휘자가 흐름을 조율하다	69
4.4 order-service : 동기 호출 제거, 이벤트 발행	73
4.5 product-service · delivery-service : Kafka Consumer/Producer 추가	77
4.6 orchestrator : 워크플로우 조율 서비스 구현 (핵심)	78
4.7 Kubernetes : Kafka와 orchestrator 배포	84
4.8 실행 및 결과 확인	85
정리	90
챕터 5. 실시간 알림 – 주문 완료를 즉시 전달하다	92
5.1 챕터 4가 남긴 두 가지 숙제	92
5.2 WebSocket : 폴링의 한계를 넘다	93
5.3 배달 완료 라이프사이클 설계	93
5.4 delivery-service : 배달 완료 API 추가	94
5.5 orchestrator : delivery-completed 처리 추가	97
5.6 order-service : STOMP로 실시간 Push 구현	99
5.7 전체 시스템 통합 테스트	103
정리	111
마치며	112
챕터별 비교	112
마지막으로	112

챕터 0. 프롤로그

0.1 금요일 밤, 서버가 멈추다.

금요일 저녁 7시였습니다.

한 주를 마무리하며 노트북을 닫으려던 찰나, 정적을 깨는 알람 소리와 함께 메신저에 팀장님의 이름이 떠올랐습니다.

“오픈 씨, 지금 로그인이 왜 이렇게 느려? 고객센터에 전화가 오기 시작했어.”

가슴이 철렁 내려앉았습니다. 서둘러 확인해 보니 주문 요청이 폭주하면서 서버의 자원을 통째로 잡아먹고 있었습니다. 더 큰 문제는 주문 기능만 느린 게 아니었다는 점입니다. 로그인도, 상품 목록 조회도, 심지어 마이페이지까지 모든 기능이 약속이라도 한 듯 함께 멈춰버렸습니다.

원인은 명확했습니다. 회원, 상품, 주문, 배달까지 모든 기능이 하나로 묶여 있는 ‘모놀리식(Monolithic)’ 구조였기 때문입니다. 하나가 무너지면 나머지도 같이 무너질 수밖에 없는 구조였지요. 주문 트래픽만 감당하기 위해 서버를 늘리고 싶어도, 결국 전체 시스템을 통째로 복제해야 하는 비효율적인 상황이었습니다.

어떻게든 야근으로 상황을 수습했지만, 가슴 한구석은 여전히 답답했습니다. 다음에 또 트래픽이 몰리면 똑같은 비극이 반복될 게 뻔했으니까요. 월요일 아침, 저는 선배에게 지난 금요일의 악몽 같은 이야기를 털어놓았습니다.

선배는 커피를 한 모금 마시더니 무심한 듯 핵심을 짚어주었습니다.

“MSA(마이크로서비스 아키텍처) 한번 알아봐. 서비스를 기능별로 쪼개서 독립적으로 돌리는 건데, 지금 너한테 딱 필요한 처방전일 거야.”

자리로 돌아와 검색창에 ‘MSA’를 입력했습니다. 개념은 이해가 갔지만, 분산 트랜잭션이니 Saga 패턴이니 하는 낯선 단어들이 쏟아지자 머릿속이 하얗게 변했습니다. 글로는 알 것 같은데, 막상 코드를 한 줄 쓰려니 손가락이 움직이지 않았습니다.

막막한 표정으로 다시 선배를 찾아갔습니다.

“선배님, 개념은 알겠는데 이걸 어떻게 코드로 옮겨야 할지 모르겠어요.”

“처음부터 완벽하게 하려고 하지 마. 일단 서비스를 나누는 것부터 시작해 봐. 가장 단순한 방법으로 말이야.”

0.2 서비스를 나누다.

선배의 조언대로 가장 단순한 형태부터 시작해 보기로 했습니다.

기능별로 네 개의 독립된 서비스를 만들고 서로를 연결했습니다. 사용자가 주문 버튼을 누르면 주문 서비스가 상품 서비스에 재고를 줄여달라 요청하고, 연이어 배달 서비스에 배달 생성을 요청하는 방식이었습니다. 각 서비스가 서로 직접 전화를 걸듯 호출하는 구조였지요. 화면에 기능들이 각자 돌아가는 것을 보니 뿌듯한 마음이 들었습니다.

하지만 곧바로 ‘데이터의 일관성’이라는 벽에 부딪혔습니다. 재고는 이미 줄였는데, 배달 서비스에서 오류가 발생한다면 어떻게 될까요? 예전처럼 하나의 서버 안에 있을 때는 문제가 생기면 자동으로 모든 작업이 취소됐지만, 이제는 각자의 데이터베이스를 가진 서비스들이라 자동으로 되돌릴 방법이 없었습니다.

“선배님, 서비스를 나누니까 트랜잭션이 하나로 안 묶여요. 중간에 실패하면 데이터가 꼬이는데 이걸 어떻게 되돌려야 하죠?”

“그럴 때 필요한 게 ‘보상 트랜잭션’이야. 어디까지 진행됐는지 기록해 뒀다가, 실패하면 마치 테이프를 감듯 역순으로 취소 작업을 실행하는 거지. 직접 한번 짜봐.”

실패 시 실행될 취소 코드를 작성하기 시작했습니다. 재고를 줄였다면 다시 늘려주고, 배달을 만들었다면 취소하는 식이었죠. 생각보다 코드는 지저분해졌습니다. 서비스가 하나 늘어날 때마다 되돌리는 코드도 함께 늘어나야 했으니까요.

그래도 동작은 확실했습니다. 품질 상품을 주문하면 에러가 나고, 이미 줄였던 재고가 자동으로 복구되는 것을 확인했을 때의 그 짜릿함은 잊을 수 없습니다.

그런데 며칠 뒤, 또 다른 허점이 발견되었습니다. 상품 서비스가 조금만 느려져도 주문 서비스까지 덩달아 거북이걸음이 되었고, 상품 서비스가 죽으면 주문 서비스마저 멈춰버렸습니다. 서비스를 나누긴 했지만, 여전히 보이지 않는 끈에 강하게 묶여 있는 느낌이었습니다.

“선배님, 서비스를 나눴는데도 하나가 죽으면 나머지도 같이 죽어버려요.”

제 물음에 선배가 웃으며 대답했습니다.

“서로 직접 전화를 걸어서 기다려주니까 그렇지. 이제 전화를 끊고 ‘메시지’를 남기는 비동기 방식으로 바꿔봐. **Kafka**(카프카)를 한번 공부해 보는 건 어때?”

0.3 비동기로 전환하다.

Kafka를 공부하며 통신 방식을 완전히 바꾸기 시작했습니다.

이제 서비스들은 서로 직접 호출하지 않습니다. 대신 게시판에 메시지를 남기듯 “주문이 들어왔어”라고 공고를 올리고 바로 자기 할 일을 하러 돌아갑니다. 그러면 상품 서비스는 자기 속도에 맞춰 메시지를 확인한 뒤 재고를 줄이고, 결과를 다시 메시지로 남기는 식이었죠.

하지만 이 복잡한 메시지들의 흐름을 누군가는 지켜봐야 했습니다. 재고가 무사히 줄었는지 확인해서 다음 단계로 넘겨야 하고, 실패했다면 보상 트랜잭션을 실행하라고 지시해야 했으니까요.

“메시지 방식으로 바꾸니까 전체 흐름을 누가 관리해야 할지 모르겠어요.”

“그럴 때 ‘오케스트레이터’를 만드는 거야. 각 서비스는 자기 일만 하고 결과만 보고하게 해. 그러면 오케스트레이터가 전체 상황을 지휘하며 다음 단계를 결정하는 거지. 실패했을 때 되돌리는 지휘봉도 애가 휘두르는 거고.”

저는 전체 흐름을 조율하는 ‘오케스트레이터’ 서비스를 만들었습니다. 메시지를 읽고, 각 주문이 어디까지 진행됐는지 추적하며 다음 단계를 지시하는 관제탑 같은 존재였지요.

실험을 위해 상품 서비스를 일부터 꺼보았습니다. 예전 같으면 주문 서비스까지 멈췄겠지만, 이제는 달랐습니다. 주문 서비스는 멀쩡히 돌아갔고 메시지는 Kafka에 안전하게 쌓였습니다. 잠시 후 상품 서비스를 다시 켜자, 밀려 있던 메시지들이 순서대로 처리되기 시작했습니다.

단순히 서비스를 나눈 것을 넘어, 진정한 독립을 이룬 기분이었습니다.

0.4 실시간 알림을 추가하다.

서비스 분리도, 비동기 통신도 성공적이었습니다. 하지만 실제 사용자 입장에서 써보니 한 가지 아쉬운 점이 남았습니다.

주문 버튼을 눌러도 즉시 완료되지 않고, 메시지가 오가는 동안 “처리 중”이라는 상태에 머물러야 했기 때문입니다. 주문이 정말 끝났는지 확인하려면 사용자가 직접 새로고침 버튼을 눌러야만 했습니다. 배달 역시 기사님이 물건을 전달한 순간, 사용자의 화면에 즉시 나타나지 않았지요.

“사용자가 새로고침을 누르지 않아도, 주문이나 배달 완료 소식을 바로 알게 하고 싶습니다.”

“그럼 WebSocket(웹소켓)을 붙여봐. 서버가 먼저 사용자에게 말을 걸 수 있게 통로를 열어주는 거야.”

마지막 퍼즐인 WebSocket을 도입했습니다. 이제 배달 기사님이 완료 버튼을 누르는 순간, 오케스트레이터의 명령을 받은 시스템이 사용자의 화면으로 즉각 알림을 쏘아 올립니다.

직접 주문을 넣어보았습니다. 화면에 “처리 중”이 표시되었고, 제가 배달 완료 신호를 보내자 1초도 안 되어 화면이 저절로 바뀌었습니다. “주문 완료!” 새로고침을 누르지 않았는데도 시스템이 살아 움직이며 저에게 말을 걸어온 것입니다.

완성된 시스템을 선배에게 보여주었습니다.

“금요일 밤에 서버가 터졌을 때는 정말 막막했는데, 한 단계씩 부딪히며 오다 보니 결국 해내게 되네요.”

선배는 흐뭇한 표정으로 제 모니터를 바라보며 말했습니다.

“처음부터 모든 정답을 알고 시작하는 사람은 없어. 부딪히고, 고치고, 다시 만드는 것. 그게 개발자에게는 가장 빠른 길이란단.”

이 여정에서 만나게 될 것들

이 책은 오픈이가 걸어온 길을 그대로 따라갑니다. 하나의 쇼핑몰 주문 시스템이 다섯 개의 챕터를 거치며 단계적으로 진화합니다.

1장 — MSA란 무엇인가? 모놀리식 구조가 왜 한계에 부딪히는지, 서비스를 나누면 어떤 점이 달라지는지를 알아봅니다. 이 책 전체를 관통하는 핵심 과제인 분산 트랜잭션과 Saga 패턴의 개념을 먼저 잡고 갑니다.

2장 — 동기식 MSA 구현 4개 서비스를 직접 만들고 REST로 연결합니다. “재고는 줄었는데 배달이 실패하면?”이라는 질문에 답하기 위해, 보상 트랜잭션을 직접 코드로 작성합니다. 단순하지만 확실하게 동작하는 첫 번째 MSA를 완성합니다.

3장 — 클린 아키텍처와 Kubernetes 2장 코드의 아쉬운 점을 UseCase 인터페이스로 개선하고, H2 대신 MySQL을 연결합니다. 그리고 Kubernetes 위에 올려서 실제 운영 환경을 처음으로 경험합니다.

4장 — 비동기 MSA, Kafka 서비스끼리 직접 전화를 거는 동기 방식을 걷어내고, Kafka 메시지로 소통하는 비동기 방식으로 전환합니다. 오케스트레이터가 전체 흐름을 지휘하고, 실패 시 자동으로 롤백하는 Orchestration Saga를 구현합니다.

5장 — 실시간 알림, WebSocket 배달 기사가 완료 버튼을 누르는 순간, 사용자 화면에 알림이 뜨는 시스템을 완성합니다. WebSocket으로 서버가 먼저 클라이언트에게 말을 걸 수 있게 됩니다. 이 장을 마치면 처음 구상했던 쇼핑몰 주문 시스템이 완성됩니다.

자, 그럼 1장에서 만나겠습니다.

개발 환경 준비

실습을 시작하기 전에 아래 도구를 설치해주세요.

도구	버전	용도
Java	21	애플리케이션 실행
IntelliJ IDEA	최신 버전	코드 작성
Docker Desktop	최신 버전	컨테이너 실행
Kubernetes	Docker Desktop	K8s 실습 (3장~)

Gradle과 Spring Boot는 프로젝트 내부에 포함되어 있으므로 별도 설치가 필요 없습니다. 각 챕터 시작에도 해당 챕터의 GitHub 주소가 안내됩니다.

프로젝트의 완성된 소스코드는 <https://github.com/metacoding-12-msa/msa> 에서 확인할 수 있습니다. 각 챕터 시작에 안내되는 GitHub 주소는 해당 챕터의 실습용 코드입니다. 각 챕터의 GitHub 저장소를 **Fork** 하여 자신의 계정에 복사한 뒤, 로컬에 **Clone** 하여 실습을 진행합니다.

챕터 1. MSA란 무엇인가?

학습 목표

- 모놀리식 아키텍처의 한계를 이해한다.
- 마이크로서비스 아키텍처가 어떤 방식으로 문제를 해결하는지 이해한다.
- 이 책에서 만들 쇼핑몰 주문 시스템의 전체 구조를 파악한다.
- MSA의 핵심 과제인 분산 트랜잭션과 Saga 패턴을 이해한다.
- 챕터 2~5의 학습 흐름을 파악한다.

1.1 모놀리식 : 쇼핑몰을 하나의 서버로 만들면 어떻게 될까?

1.1.1 처음에는 아무 문제가 없었다

백화점을 떠올려 보세요. 수십 개의 매장이 한 건물 안에 모여 있습니다. 고객은 한 곳에서 모든 것을 해결할 수 있고, 운영사 입장에서는 전기·냉방·보안·고객 데이터를 한 곳에서 통합 관리합니다. 이 구조는 단순하고 효율적입니다.



그림 0-1: 백화점 — 모든 매장이 한 건물에 모여 있다

소프트웨어도 같은 이야기입니다. 처음에는 하나의 서버에 모든 기능을 넣는 모놀리식 구조가 단순하고 빠릅니다.

1.1.2 성장하면서 균열이 생긴다

백화점이 잘 돼서 방문객이 열 배로 늘었습니다. 이제 문제가 보이기 시작합니다.

[백화점의 한계]

문제1: 한 매장에서 화재 발생

→ 스프링클러 작동·대피령으로 전 층 영업 중단

문제2: 전자제품 세일로 3층에 사람이 몰림

→ 3층만 확장 불가, 건물 전체를 새로 지어야 함

문제3: 건물 전기 배선 전면 교체

→ 공사 기간 동안 전 층 영업 중단

소프트웨어 세계에서든 똑같은 일이 벌어집니다. 회원 10만 명, 하루 주문 1만 건이 되었을 때, 모놀리식 구조의 균열이 드러납니다.

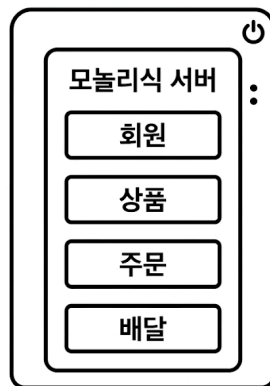


그림 0-2: 모놀리식 쇼핑몰 – 모든 기능이 하나의 서버에

배포가 두렵다 – 주문 기능 하나를 수정해도 배포 시 회원·상품·배달까지 전부 재시작해야 합니다.

장애가 전파된다 – 배달 기능 버그로 서버가 느려지면 배달과 무관한 회원 로그인도 함께 느려집니다.

확장이 어렵다 – 블랙프라이데이에 주문 기능만 서버를 늘리고 싶어도 모놀리식에서는 전체 서버를 복제해야 합니다.

팀이 커지면 충돌이 잦아진다 – 10명이 하나의 코드베이스를 동시에 수정하면 하루에도 수십 번 충돌이 발생합니다.

1.2 마이크로서비스 : 역할을 나눈다

1.2.1 백화점 vs 개별 상점

개별 상점 방식은 백화점과 구조 자체가 다릅니다. 각 매장이 독립된 건물로 운영되어, 자신만의 전기·냉방·입구를 가집니다.

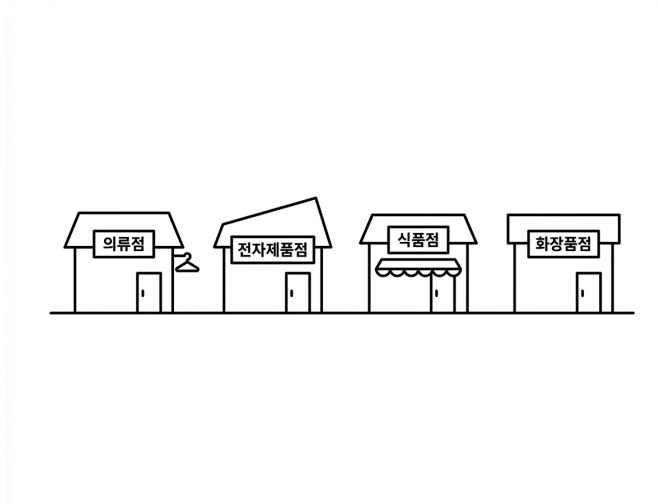


그림 0-3: 개별 상점 – 각 매장이 독립된 건물로 운영

백화점 구조와 비교하면 차이가 분명합니다.

	백화점	개별 상점
한 매장 화재 발생	스프링클러·대피령으로 전 층 영업 중단	해당 매장만 소방 처리, 나머지 정상 영업
전자제품 수요 폭증	건물 전체를 확장해야 함	전자제품점만 확장
건물 전기 배선 교체	공사 기간 동안 전 층 영업 중단	해당 매장만 임시 폐쇄, 나머지 정상 영업

마이크로서비스 아키텍처가 바로 이 방식입니다. 하나의 큰 서버 대신, 기능별로 작은 서비스들을 분리합니다.

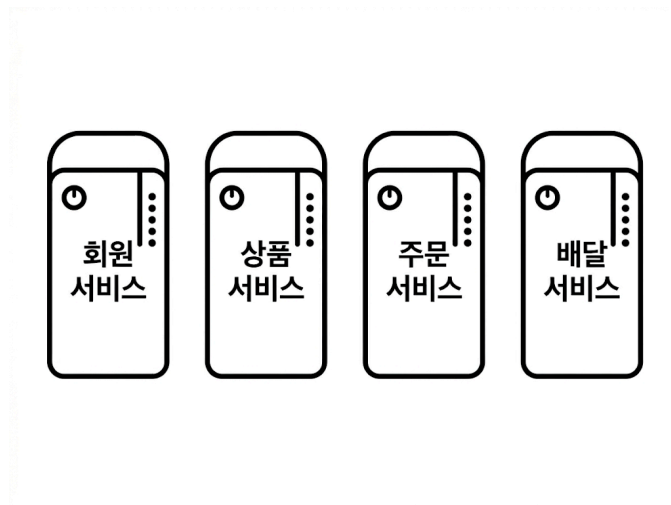


그림 0-5: 마이크로서비스 쇼핑물 – 기능별로 서비스를 분리

이제 각 서비스는 독립적으로 배포하고, 독립적으로 확장할 수 있습니다. 주문 서비스에 버그가 생겨도 회원 서비스는 영향을 받지 않습니다.

1.2.2 MSA vs 모놀리식

특성	모놀리식	마이크로서비스
배포	전체 재배포	해당 서비스만 배포
장애 격리	전체 영향	해당 서비스만 영향
확장	전체 복제	필요한 서비스만 확장
팀 분리	하나의 코드베이스	서비스별 독립 개발

1.3 시스템 설계 : 우리가 만들 서비스 구조

문제를 이해했으니, 이제 직접 만들어볼 시스템을 설계해보겠습니다. 이 책의 쇼핑물 주문 시스템은 4개의 마이크로서비스로 구성됩니다.

서비스	포트	역할
회원 서비스	8083	로그인, JWT 발급, 사용자 조회
상품 서비스	8082	상품 목록, 재고 조회 및 증감
주문 서비스	8081	주문 생성·조회·취소 (핵심)
배달 서비스	8084	배달 생성·조회·취소

1.3.1 챕터 2~3 아키텍처 (동기 통신)

챕터 2와 챕터 3에서 만들 시스템입니다. 주문 서비스가 중심에서 다른 서비스를 직접 REST API로 호출합니다.

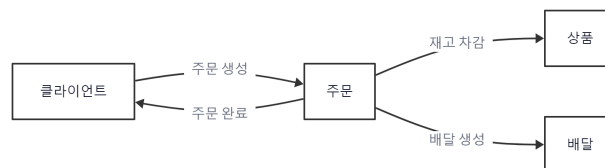


그림 0-8: 챕터 2~3 아키텍처 – 동기 REST 통신

사용자가 주문을 생성하면 주문 서비스가 상품 서비스와 배달 서비스를 RestClient로 차례로 호출합니다. 응답을 받을 때까지 기다리는 동기 방식입니다. 이 구조를 먼저 직접 구현해보면, 나중에 비동기 방식으로 전환했을 때 그 차이가 더 명확하게 느껴집니다.

1.3.2 챕터 4~5 아키텍처 (비동기 통신)

챕터 4와 챕터 5에서는 직접 호출을 건너내고 Kafka를 도입합니다. 중앙의 오케스트레이터가 전체 흐름을 조율하고, 각 서비스는 Kafka를 통해 재고 차감, 배달 생성 메시지를 주고받습니다.



그림 0-9: 챕터 4~5 아키텍처 – Kafka 비동기 통신

주문 서비스는 주문을 저장하고 Kafka에 이벤트만 발행한 뒤 즉시 응답합니다. 오케스트레이터가 나머지 흐름을 이어받아 처리하므로, 서비스 간 직접 연결이 사라집니다. 이 구조의 장점은 챕터 4에서 직접 체험합니다.

1.4 분산 트랜잭션 : MSA의 핵심 과제

서비스를 분리하면 좋은 점이 많지만, 동시에 새로운 문제가 생깁니다. 바로 분산 트랜잭션입니다. 이 개념을 이해하는 것이 이 책의 핵심입니다.

1.4.1 모놀리식에서는 쉬웠던 것

모놀리식에서는 데이터베이스 트랜잭션이 간단합니다. 주문, 재고 변경, 배달 생성을 하나의 `@Transactional` 블록 안에 넣으면, 하나라도 실패하면 전부 자동 롤백됩니다.

모놀리식에서 단일 트랜잭션으로 처리하는 예시입니다.

[참고] 동작 이해용입니다.

```
// 모놀리식: 하나의 트랜잭션으로 처리 가능
@Transactional
public void createOrder() {
    decreaseStock(); // 재고 감소
    createDelivery(); // 배달 생성
    saveOrder();     // 주문 저장
    // 실패 시 세 가지 모두 자동 롤백
}
```

1.4.2 MSA에서는 불가능하다

하지만 MSA에서는 각 서비스가 독립된 데이터베이스를 가집니다. DB를 공유하면 한 서비스의 테이블 변경이 다른 서비스에 영향을 주고, 배포와 확장도 함께 묶이기 때문입니다. 상품 서비스의 DB와 배달 서비스의 DB가 물리적으로 분리되어 있으므로, 하나의 트랜잭션으로 묶을 방법이 없습니다.

이 책에서는 학습 편의를 위해 하나의 MySQL 인스턴스를 공유합니다. 실무에서는 서비스별로 DB를 분리하는 것이 원칙입니다.

이것은 서로 다른 은행 간 송금과 비슷합니다. 같은 은행 안에서 이체하면 A 계좌에서 빠진 돈이 B 계좌로 즉시 들어가고, 문제가 생기면 자동으로 원상 복구됩니다. 하지만 다른 은행으로 송금할 때는 이야기가 달라집니다. A 은행에서 돈은 빠졌는데 B 은행에 입금이 실패하면, A 은행이 자동으로 알 수 없습니다. 별도의 확인과 복구 절차가 필요합니다.

분산 트랜잭션(Distributed Transaction): 여러 독립된 데이터베이스에 걸친 작업을 하나의 논리적 단위로 처리해야 하는 상황입니다. MSA에서는 서비스마다 DB가 분리되어 있으므로 단일 트랜잭션이 불가능하고, 별도의 전략이 필요합니다.

배달 생성이 실패했을 때, 이미 감소된 재고를 어떻게 되돌릴까요? 상품 서비스와 배달 서비스의 DB가 분리되어 있으므로, 자동 롤백이 불가능합니다.

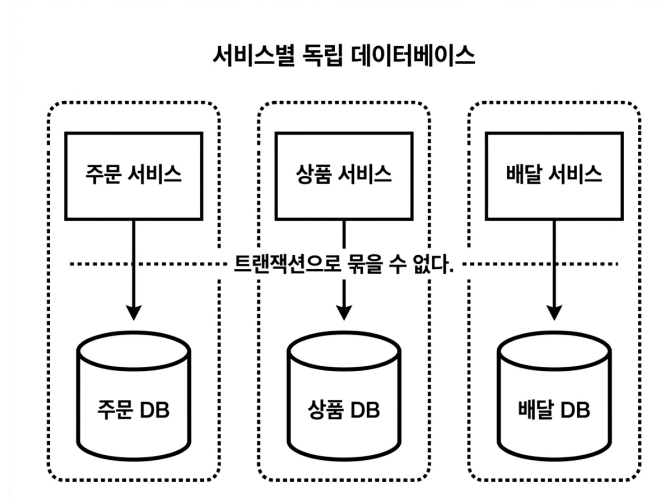


그림 0-10: 서비스별 독립 데이터베이스

1.4.3 해결 방법: Saga 패턴

이 문제를 어떻게 해결할까요? 다시 은행 비유로 돌아가 봅시다. 타행 송금이 실패하면 은행은 “송금 실패 → 출금액 환불”이라는 정해진 절차를 밟습니다. 자동 롤백은 안 되지만, 실패를 감지하면 단계별로 되돌리는 것입니다. 이 책에서는 이 접근법을 **Saga 패턴**으로 구현합니다.

Saga 패턴: 분산 트랜잭션을 여러 개의 로컬 트랜잭션으로 나누고, 중간에 실패가 발생하면 이전 단계를 역순으로 취소(보상)하여 전체 정합성을 맞추는 패턴입니다.

두 가지 방식을 순서대로 배웁니다.

Choreography Saga — 챕터 2, 챕터 3

동네 가게들이 서로 전화로 직접 소통하는 방식입니다. 꽃집이 케이크 가게에 직접 전화해서 “케이크 취소해주세요”라고 말하듯, 실패가 발생하면 이전 단계 서비스에 직접 복구를 요청합니다. 주문 서비스가 직접 상품 서비스에 “재고 돌려줘”라고 요청합니다.

Choreography Saga(코레오그래피 사가): 중앙 조율자 없이 각 서비스가 서로 직접 호출하여 트랜잭션을 이어가고, 실패 시 이전 서비스에 직접 보상(복구)을 요청하는 방식입니다.

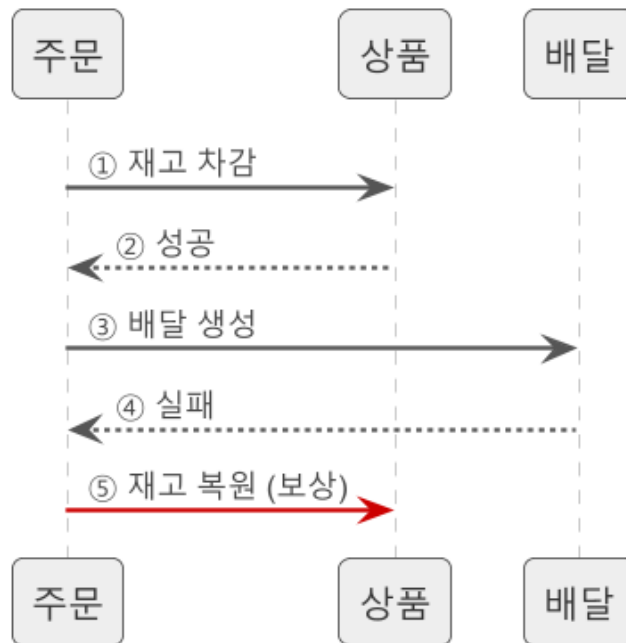


그림 0-11: Choreography Saga – 서비스 간 직접 호출과 보상

서비스끼리 서로 직접 호출해서 복구하는 방식입니다. 단순하지만, 서비스 수가 늘어날수록 복잡해집니다.

Orchestration Saga – 챕터 4, 챕터 5

이번에는 웨딩 플래너를 떠올려 봅시다. 결혼식에는 꽃집, 케이크 가게, 사진작가, 밴드 등 여러 업체가 참여합니다. 신랑 신부가 각 업체에 일일이 전화하는 대신, 웨딩 플래너가 전체 일정을 조율합니다. 한 업체에 문제가 생기면 플래너가 나머지 업체에 변경 사항을 알립니다.

별도의 오케스트레이터가 이 웨딩 플래너 역할을 합니다. 전체 흐름을 조율하고, 각 서비스는 자신의 일만 하고 Kafka로 결과를 발행합니다. 오케스트레이터가 결과를 받아 다음 단계를 결정합니다.

Orchestration Saga(오케스트레이션 사가): 중앙의 조율자(오케스트레이터)가 전체 트랜잭션 흐름을 관리하고, 각 서비스에 명령을 내리고 결과를 받아 다음 단계를 결정하는 방식입니다. 실패 시 오케스트레이터가 자동으로 보상 명령을 내립니다.

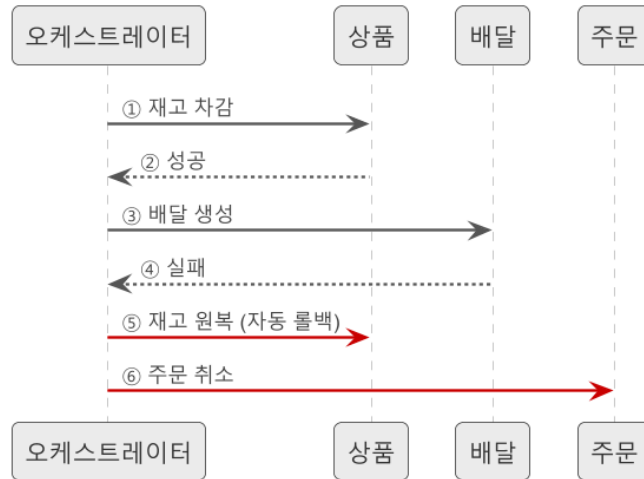


그림 0-12: Orchestration Saga — 오케스트레이터가 흐름을 조율

오케스트레이터가 전체 흐름을 알고 있기 때문에, 실패 시 자동으로 롤백 명령을 내립니다.

두 방식을 모두 직접 구현해보면, 각각의 장단점이 자연스럽게 체감됩니다.

1.5 이 책의 학습 흐름

이제 전체 그림이 보입니다. 이 책은 하나의 시스템이 단계별로 진화하는 여정입니다. 각 챕터는 이전 챕터의 한계를 느끼는 것에서 시작합니다.

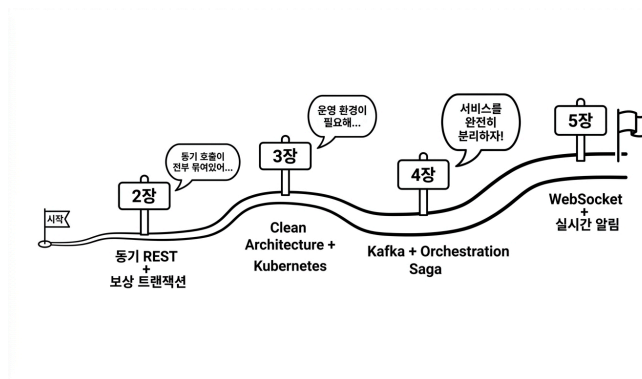


그림 0-13: 이 책의 학습 흐름

챕터 2 – 4개 서비스를 REST로 연결하고 보상 트랜잭션을 구현합니다. MSA의 뼈대를 직접 손으로 만드는 챕터입니다.

챕터 3 – 챕터 2 코드의 아쉬운 점을 클린 아키텍처로 개선하고, Kubernetes에 올려 운영 환경을 경험합니다.

챕터 4 – 동기 REST 호출의 한계를 Kafka로 해결합니다. 서비스가 완전히 분리되는 경험을 합니다.

챕터 5 – 배달 기사가 완료 API를 호출하는 순간, 사용자 화면에 실시간 알림이 뜨는 시스템을 완성합니다.

■ 정리

- 모놀리식은 처음에는 단순하지만, 서비스가 커지면 배포·장애·확장 문제가 생깁니다.
- 마이크로서비스는 기능별로 서비스를 분리하여 각자 독립적으로 배포하고 확장할 수 있게 합니다.
- MSA의 핵심 과제는 분산 트랜잭션입니다. 각 서비스의 DB가 분리되어 있어 단일 트랜잭션으로 묶을 수 없습니다.
- **Saga** 패턴으로 분산 트랜잭션을 해결합니다. Choreography Saga(챕터 2~3)와 Orchestration Saga(챕터 4~5) 두 가지를 이 책에서 배웁니다.
- 이 책은 하나의 쇼핑몰 주문 시스템이 챕터 2부터 챕터 5까지 단계적으로 진화하는 이야기입니다.

이제 직접 코드를 작성할 시간입니다. 챕터 2에서는 4개의 서비스를 REST로 연결하고, 보상 트랜잭션을 구현해봅니다. 처음에는 단순하게 시작합니다. 그 단순함이 나중에 어떤 문제를 만드는지 직접 느껴보는 것이 챕터 2의 핵심입니다.

챕터 2. 동기식 MSA 구현 - 서비스를 연결하다

chap01 . 실행 환경: 로컬 · 4개 서비스 · H2 DB 이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/chap01> 에서 확인할 수 있습니다.

학습 목표

- Spring Boot로 4개 서비스를 독립적으로 실행하고 REST로 연결한다.
- JWT 인증 필터를 구현하고 서비스 간 Authorization 헤더를 전달한다.
- 주문 생성 시 재고 감소 → 배달 생성 흐름을 동기적으로 구현한다.
- 중간에 실패가 발생했을 때 이전 작업을 되돌리는 보상 트랜잭션을 작성한다.

2.1 이야기의 시작 - 네 개의 서비스가 만나다

챕터 1에서 설계한 네 개의 서비스(회원, 상품, 주문, 배달)를 이제 직접 만들어 보겠습니다.

이번 챕터의 주인공은 주문 서비스입니다. 주문을 생성하려면 상품 서비스에서 재고를 줄이고, 배달 서비스에서 배달을 만들어야 합니다. 즉 주문 서비스가 두 서비스를 직접 호출해야 합니다. 그런데 여기서 중요한 문제가 생깁니다. “재고는 줄였는데 배달 생성이 실패하면 어떻게 해야 할까?” 이 질문에 대한 답이 바로 이번 챕터의 핵심, 보상 트랜잭션입니다.

보상 트랜잭션(Compensating Transaction): 여러 서비스에 걸친 작업 중 일부가 실패했을 때, 이미 완료된 작업을 되돌리기 위해 역순으로 실행하는 취소 작업입니다.

2.1.1 서비스별 독립 Gradle 프로젝트

각 서비스는 독립된 Gradle 프로젝트입니다. 하나의 서비스를 배포할 때 다른 서비스를 건드릴 필요가 없습니다.

```
chap01/
├── user/      # 포트 8083
```

```

├─ product/      # 포트 8082
├─ order/        # 포트 8081
├─ delivery/     # 포트 8084
└─ docker-compose.yml # 전체 서비스 실행

```

각 서비스 내부 패키지 구조는 아래와 같습니다. core 패키지에는 JWT, 예외 처리, 표준 응답 등 모든 서비스에 공통으로 필요한 코드가 들어갑니다. 주문 서비스 기준으로 보여드리며, 회원/상품/배달 서비스도 동일한 구조입니다.

```

src/main/java/com/metacoding/order/
├─ OrderApplication.java      # [참고]
├─ core/
│ └─ config/
│   │ └─ WebConfig.java      # [참고] JWT 필터 등록
│   │ └─ RestClientConfig.java # [작성] JWT 헤더 전달 인터셉터
│   └─ filter/
│     └─ JwtAuthenticationFilter.java # [참고] JWT 인가 필터
│   └─ handler/
│     └─ GlobalExceptionHandler.java # [참고] 전역 예외 처리
│     └─ ex/                  # 커스텀 예외 (Exception400~500)
│   └─ util/
│     └─ JwtProvider.java     # [참고] JWT 파싱/검증
│     └─ JwtUtil.java         # [참고] JWT 생성
│     └─ Resp.java           # [참고] 표준 응답 래퍼
├─ order/
│ └─ Order.java              # [참고] JPA 엔티티
│ └─ OrderItem.java          # [참고] JPA 엔티티
│ └─ OrderController.java    # [참고] REST 컨트롤러
│ └─ OrderService.java       # [작성] 비즈니스 로직
│ └─ OrderRepository.java    # [참고] Spring Data JPA
│ └─ OrderItemRepository.java # [참고] Spring Data JPA
│   └─ OrderRequest.java / OrderResponse.java # [참고]
└─ adapter/                  # 주문 서비스에만 존재
    └─ ProductClient.java    # [작성] 상품 서비스 호출
    └─ DeliveryClient.java   # [작성] 배달 서비스 호출
    └─ dto/                  # 어댑터용 DTO
Dockerfile                  # [참고] Docker 이미지 빌드

```

참고: 회원/상품/배달 서비스는 `adapter/` 패키지와 `RestClientConfig` 가 없고, 나머지 구조는 동일합니다.

2.2 공통 설정 : 모든 서비스가 공유하는 뼈대

서비스 구조를 잡았으니, 먼저 공통 기반을 만들겠습니다. MSA에서는 서비스마다 서버가 다르므로 세션을 공유할 수 없습니다. 대신 JWT 토큰을 발급하고, 각 서비스가 토큰만 검증하는 방식을 사용합니다. 4개 서비스 모두 JWT 인증, 표준 응답 형식, 예외 처리가 필요합니다. 이를 `core/` 패키지에 모아 두면, 각 서비스는 비즈니스 로직에 집중할 수 있습니다.

각 컴포넌트의 역할은 다음과 같습니다.

컴포넌트	역할
JwtAuthenticationFilter	매 요청마다 <code>Authorization</code> 헤더에서 JWT를 꺼내 검증하고, 토큰 안의 <code>userId</code> 를 request attribute에 저장합니다. 컨트롤러는 <code>@RequestAttribute("userId")</code> 로 현재 사용자를 식별합니다.
JwtUtil / JwtProvider	JwtUtil은 로그인 성공 시 JWT를 생성하고, JwtProvider는 요청에서 받은 토큰을 파싱·검증합니다.
Resp	모든 API 응답을 <code>{status, msg, body}</code> 형태로 통일하는 래퍼 클래스입니다. 성공과 실패 모두 같은 구조로 반환하여 클라이언트 파싱을 단순화합니다.
GlobalExceptionHandler	<code>@RestControllerAdvice</code> 로 전역 예외를 잡아 Resp 형태의 에러 응답을 반환합니다.
WebConfig	JWT 필터를 등록합니다. <code>/api/*</code> 경로에 <code>JwtAuthenticationFilter</code> 를 적용하여 인증이 필요한 요청을 필터링합니다.

공통 설정이 준비되었습니다. 주문 서비스의 보상 트랜잭션을 직접 작성하기 전에, 나머지 세 서비스의 핵심 코드를 먼저 살펴보겠습니다. 아래 2.3~2.5는 [참고] 코드로, 동작 이해를 위해 주요 부분만 보여줍니다.

전체 코드는 GitHub에서 확인할 수 있습니다.

2.3 회원 서비스 : JWT로 로그인하다

회원 서비스는 로그인과 사용자 조회를 담당합니다. 사용자가 `POST /login` 으로 아이디와 비밀번호를 보내면, 회원 서비스가 DB에서 조회하고 비밀번호를 검증합니다. 검증에 성공하면 JWT 토큰을 응답 헤더에 넣어 돌려줍니다. 이 토큰이 이후 모든 서비스 요청의 인증 수단이 됩니다.

메서드	경로	기능
POST	/login	로그인 (JWT 발급)
GET	/api/users/{userId}	사용자 조회

2.3.1 User 엔티티

`username` 은 유니크 제약으로 중복 가입을 방지합니다.

[참고] `users/User.java`

```
@NoArgsConstructor
@Getter
@Entity
@Table(name = "user_tb")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    @Column(unique = true)
    private String username;
    private String email;
    private String password;
    private String roles;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @Builder
    private User(String username, String email, String password, String roles) {
        this.username = username;
        this.email = email;
        this.password = password;
    }
}
```

```

    this.roles = roles;
    this.createdAt = LocalDateTime.now();
    this.updatedAt = LocalDateTime.now();
  }
}

```

2.3.2 더미 데이터

개발 환경에서는 H2 in-memory DB에 자동으로 테스트 데이터를 삽입합니다. [db/data.sql](#) 에 ssar, cos, love 세 계정이 등록됩니다.

[참고] [resources/db/data.sql](#)

```

INSERT INTO user_tb (username, email, password, roles, created_at, updated_at)
VALUES ('ssar','ssar@metacoding.com','1234','USER',now(),now());

INSERT INTO user_tb (username, email, password, roles, created_at, updated_at)
VALUES ('cos','cos@metacoding.com','1234','USER',now(),now());

INSERT INTO user_tb (username, email, password, roles, created_at, updated_at)
VALUES ('love','love@metacoding.com','1234','USER',now(),now());

```

2.4 상품 서비스 : 재고를 관리하다

상품 서비스는 상품 목록 조회와 재고 증감을 담당합니다. 주문 서비스가 주문을 생성할 때 이 서비스의 재고 감소 API를 호출하고, 주문이 취소되거나 실패하면 재고 증가 API로 되돌립니다.

메서드	경로	기능
GET	/api/products/{productId}	상품 조회
PUT	/api/products/{productId}/decrease	재고 감소
PUT	/api/products/{productId}/increase	재고 증가

2.4.1 Product 엔티티

[decreaseQuantity](#) 와 [increaseQuantity](#) 메서드로 재고 증감 로직이 엔티티에 캡슐화되어 있습니다.

[참고] [products/Product.java](#)

```

@NoArgsConstructor
@Getter
@Entity
@Table(name = "product_tb")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String productName;
    private int quantity;
    private Long price;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @Builder
    private Product(String productName, int quantity, Long price) {
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    public void decreaseQuantity(int quantity) {
        this.quantity -= quantity;
        this.updatedAt = LocalDateTime.now();
    }

    public void increaseQuantity(int quantity) {
        this.quantity += quantity;
        this.updatedAt = LocalDateTime.now();
    }
}

```

2.4.2 ProductService

재고 감소 전 상품 존재 여부와 가격 일치 여부를 검증합니다.

[참고] [products/ProductService.java](#)

```

@Transactional
public ProductResponse decreaseQuantity(int productId, int quantity, Long price) {
    Product findProduct = productRepository.findById(productId)

```



```

        .orElseThrow(() -> new Exception404("상품이 없습니다.));
    if (findProduct.getQuantity() < quantity) {
        throw new Exception400("상품 재고가 부족합니다.");
    }
    if (!price.equals(findProduct.getPrice())) {
        throw new Exception400("상품 가격이 일치하지 않습니다.");
    }
    findProduct.decreaseQuantity(quantity); // 재고 감소
    return ProductResponse.from(findProduct);
}

```

2.4.3 더미 데이터

`db/data.sql` 에 MacBook Pro(재고 10, 250만원), iPhone 15(재고 0 품절, 130만원), AirPods(재고 10, 30만원)가 등록됩니다. 2.7 실행 시나리오에서 품절 상품으로 주문했을 때 보상 트랜잭션이 어떻게 동작하는지 확인합니다.

[참고] [resources/db/data.sql](#)

```

INSERT INTO product_tb (product_name, quantity, price, created_at, updated_at)
VALUES ('MacBook Pro', 10, 2500000, now(), now());

INSERT INTO product_tb (product_name, quantity, price, created_at, updated_at)
VALUES ('iPhone 15', 0, 1300000, now(), now());

INSERT INTO product_tb (product_name, quantity, price, created_at, updated_at)
VALUES ('AirPods', 10, 300000, now(), now());

```

2.5 배달 서비스 : 배달을 생성하고 취소하다

배달 서비스는 배달 생성과 취소를 담당합니다. 이번 챕터에서는 배달 생성과 동시에 완료 처리합니다. PENDING 상태로 생성한 뒤 즉시 COMPLETED로 전이하므로, 실질적으로 COMPLETED 또는 CANCELLED 두 상태만 관찰됩니다.

메서드	경로	기능
POST	/api/deliveries	배달 생성
GET	/api/deliveries/{deliveryId}	배달 조회
PUT	/api/deliveries/{orderId}	배달 취소

2.5.1 Delivery 엔티티

상태는 `PENDING → COMPLETED` 또는 `PENDING → CANCELLED` 로 전이되며, `create()`, `complete()`, `cancel()` 메서드로 캡슐화되어 있습니다.

[참고] `deliveries/Delivery.java`

```
@NoArgsConstructor
@Getter
@Entity
@Table(name = "delivery_tb")
public class Delivery {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private int orderId;
    private String address;
    @Enumerated(EnumType.STRING)
    private DeliveryStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @Builder
    private Delivery(int orderId, String address, DeliveryStatus status) {
        this.orderId = orderId;
        this.address = address;
        this.status = status;
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    public static Delivery create(int orderId, String address) {
        return new Delivery(orderId, address, DeliveryStatus.PENDING);
    }
}
```

```

public void complete() {
    this.status = DeliveryStatus.COMPLETED;
    this.updatedAt = LocalDateTime.now();
}

public void cancel() {
    this.status = DeliveryStatus.CANCELLED;
    this.updatedAt = LocalDateTime.now();
}
}

```

2.5.2 DeliveryService

`createDelivery` 는 배달을 생성하고 즉시 완료 처리합니다. `cancelDelivery` 는 주문 취소 시 호출되며, 이미 취소된 배달에 대한 중복 요청을 방어합니다.

[참고] `deliveries/DeliveryService.java`

```

@Transactional
public DeliveryResponse createDelivery(int orderId, String address) {
    // 1. 배달 생성
    Delivery createdDelivery = deliveryRepository.save(Delivery.create(orderId, address));
    // 2. 주소 검증
    if (address == null || address.isBlank()) {
        throw new Exception400("배달 주소는 필수입니다.");
    }
    // 3. 배달 완료
    createdDelivery.complete();
    return DeliveryResponse.from(createdDelivery);
}

@Transactional
public DeliveryResponse cancelDelivery(int orderId) {
    Delivery findDelivery = deliveryRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("배달 정보를 조회할 수 없습니다."));
    if (findDelivery.getStatus() == DeliveryStatus.CANCELLED) {
        throw new Exception400("배달이 이미 취소되었습니다.");
    }
    findDelivery.cancel();
    return DeliveryResponse.from(findDelivery);
}

```

2.5.3 더미 데이터

주문 3건에 대한 배달 데이터가 등록됩니다. 모두 완료 상태입니다.

[참고] [resources/db/data.sql](#)

```
INSERT INTO delivery_tb (order_id, address, status, created_at, updated_at) VALUES (1, 'Addr 1',
'COMPLETED', NOW(), NOW());
INSERT INTO delivery_tb (order_id, address, status, created_at, updated_at) VALUES (2, 'Addr 2',
'COMPLETED', NOW(), NOW());
INSERT INTO delivery_tb (order_id, address, status, created_at, updated_at) VALUES (3, 'Addr 3',
'COMPLETED', NOW(), NOW());
```

2.6 주문 서비스 : 보상 트랜잭션의 현장 (핵심)

배달 서비스까지 살펴봤으니, 이제 핵심인 주문 서비스로 넘어갑니다.

메서드	경로	기능
POST	/api/orders	주문 생성
GET	/api/orders/{orderId}	주문 조회
PUT	/api/orders/{orderId}	주문 취소

먼저 주문과 주문 상품을 표현하는 엔티티부터 살펴보겠습니다.

2.6.1 Order 엔티티

상태는 `PENDING → COMPLETED` 또는 `PENDING → CANCELLED` 로 전이되며, `create()`, `complete()`, `cancel()` 메서드로 캡슐화되어 있습니다.

[참고] [orders/Order.java](#)

```
@NoArgsConstructor
@Getter
@Entity
@Table(name = "order_tb")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private int userId;
    @Enumerated(EnumType.STRING)
```

```

private OrderStatus status;
private LocalDateTime createdAt;
private LocalDateTime updatedAt;

@Builder
private Order(int userId, OrderStatus status) {
    this.userId = userId;
    this.status = status;
    this.createdAt = LocalDateTime.now();
    this.updatedAt = LocalDateTime.now();
}

public static Order create(int userId) {
    return new Order(userId, OrderStatus.PENDING);
}

public void complete() {
    this.status = OrderStatus.COMPLETED;
    this.updatedAt = LocalDateTime.now();
}

public void cancel() {
    this.status = OrderStatus.CANCELLED;
    this.updatedAt = LocalDateTime.now();
}
}

```

2.6.2 OrderItem 엔티티

하나의 주문에 여러 상품이 포함될 수 있으므로 Order와 분리합니다. `create()` 메서드로 생성합니다.

[참고] `orders/OrderItem.java`

```

@NoArgsConstructor
@Getter
@Entity
@Table(name = "order_item_tb")
public class OrderItem {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
}

```

```

private int orderId;
private int productId;
private int quantity;
private Long price;
private LocalDateTime createdAt;
private LocalDateTime updatedAt;

@Builder
private OrderItem(int orderId, int productId, int quantity, Long price) {
    this.orderId = orderId;
    this.productId = productId;
    this.quantity = quantity;
    this.price = price;
    this.createdAt = LocalDateTime.now();
    this.updatedAt = LocalDateTime.now();
}

public static OrderItem create(int orderId, int productId, int quantity, Long price) {
    return new OrderItem(orderId, productId, quantity, price);
}
}

```

2.6.3 더미 데이터

사용자별 주문 3건(완료·취소·대기)과 각 주문에 대한 주문 상품 1건씩이 등록됩니다.

[참고] <resources/db/data.sql>

```

INSERT INTO order_tb (user_id, status, created_at, updated_at) VALUES (1, 'COMPLETED', now(), now());
INSERT INTO order_tb (user_id, status, created_at, updated_at) VALUES (2, 'CANCELLED', now(), now());
INSERT INTO order_tb (user_id, status, created_at, updated_at) VALUES (3, 'PENDING', now(), now());

INSERT INTO order_item_tb (order_id, product_id, quantity, price, created_at, updated_at) VALUES (1, 1, 1, 2500000, now(), now());
INSERT INTO order_item_tb (order_id, product_id, quantity, price, created_at, updated_at) VALUES (2, 3, 1, 300000, now(), now());
INSERT INTO order_item_tb (order_id, product_id, quantity, price, created_at, updated_at) VALUES (3, 2, 2, 1300000, now(), now());

```

2.6.4 보상 트랜잭션 : 실패하면 되돌린다

엔티티가 준비되었으니, 이제 주문 서비스의 진짜 역할을 살펴봅니다. 주문 서비스는 상품 서비스와 배달 서비스를 직접 호출합니다. 그런데 서비스를 호출하면 반드시 이 질문과 마주합니다.

“재고는 줄었는데, 배달 생성이 실패하면 어떻게 하죠?”

단일 서비스였다면 DB 트랜잭션이 자동으로 롤백해줍니다. 하지만 이미 상품 서비스에 보낸 HTTP 요청은 되돌릴 수 없습니다. 우리가 직접 “재고를 다시 늘려줘”라는 HTTP 요청을 보내야 합니다. 이것이 보상 트랜잭션입니다. 챕터 1에서 소개한 Choreography Saga를 직접 HTTP 호출로 구현하는 방식입니다. 주문 서비스가 직접 각 서비스를 호출하고, 실패 시 역순으로 보상합니다.

2.6.5 보상 트랜잭션 설계

코드를 작성하기 전, 어느 단계에서 실패하면 무엇을 되돌려야 하는지 먼저 그려봅니다.

주문 생성 흐름 (단계별 보상):

1단계: 재고 감소 (상품 서비스)

└─ 실패 → 보상 없음 (아직 아무 일도 안 했으므로)

2단계: 배달 생성 (배달 서비스)

└─ 실패 → 보상: 재고 복구 (1단계 되돌리기)

3단계: 주문 완료

└─ 실패 → 보상: 배달 취소 + 재고 복구 (2, 1단계 되돌리기)

4단계: 성공 응답 반환

핵심 아이디어는 진행 상태를 플래그로 추적하는 것입니다. `productDecreased` 와 `deliveryCreated` 변수가 각각 1단계, 2단계의 성공 여부를 기록합니다. 예외가 발생했을 때 이 플래그를 보고, 어디까지 진행됐는지 확인한 뒤 역순으로 되돌립니다.

2.6.6 주문 성공 시

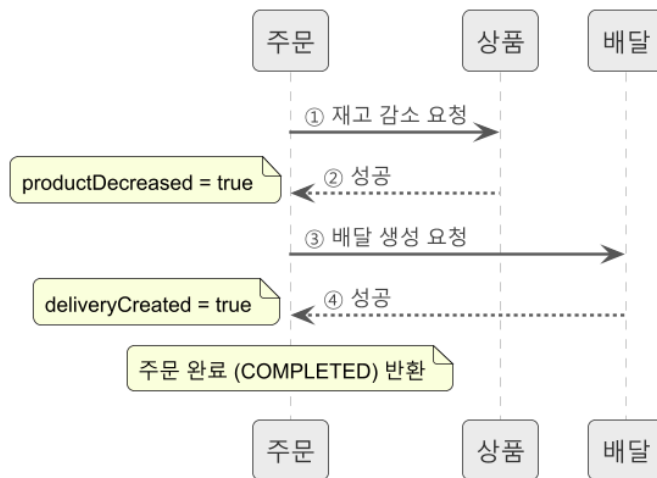


그림 0-6: 주문 성공 흐름

2.6.7 주문 실패 시

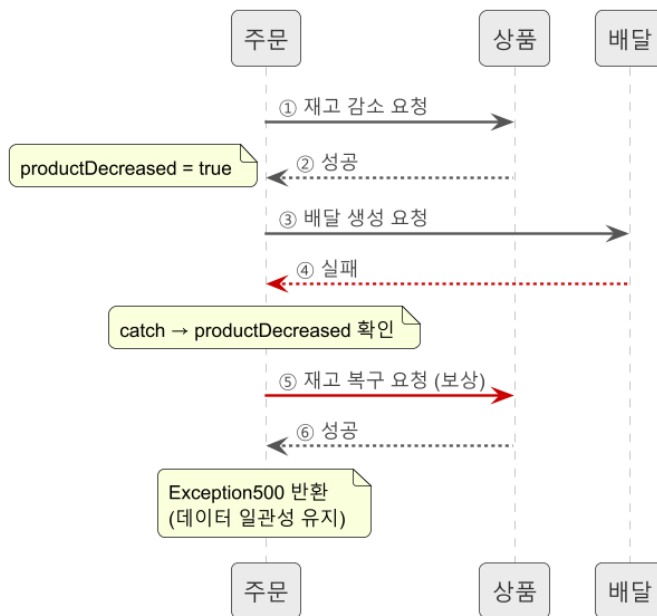


그림 0-7: 주문 실패 시 보상 트랜잭션 흐름

2.6.8 RestClient : JWT 헤더를 실어 다른 서비스를 호출하다

주문 서비스가 다른 서비스를 호출할 때 필요한 RestClient를 먼저 설정합니다. RestClient에 인터셉터(`ClientHttpRequestInterceptor`)를 등록합니다. 이렇게 하면 외부 API를 호출할 때 Authorization 헤더가 자동으로 전달되어 하위 서비스의 JWT 인증을 통과합니다.

[작성] `core/config/RestClientConfig.java` - 아래 코드를 해당 파일에 추가하세요.

```
@Configuration
public class RestClientConfig {

    @Bean
    public RestClient.Builder restClientBuilder() {
        ClientHttpRequestInterceptor authForwardingInterceptor = (request, body, execution) -> { // JWT 전달
인터셉터
            ServletRequestAttributes attributes =
                (ServletRequestAttributes) RequestContextHolder.getRequestAttributes(); // 현재 요청 정보 가져
오기
            if (attributes != null) {
                String authorization = attributes.getRequest().getHeader("Authorization"); // 원본 요청의 JWT 꺼내
기
                if (authorization != null) {
                    request.getHeaders().add("Authorization", authorization); // 나가는 요청에 JWT 실어 보내기
                }
            }
            return execution.execute(request, body);
        };

        return RestClient.builder().requestInterceptor(authForwardingInterceptor); // 인터셉터 등록한 빌더 반
환
    }
}
```

2.6.9 ProductClient와 DeliveryClient

RestClient 설정이 끝났으면, 각 서비스를 호출하는 클라이언트를 만듭니다. OrderService는 클라이언트의 메서드만 호출하면 되고, HTTP 통신 방식을 알 필요가 없습니다.

baseUrl의 `product-service`, `delivery-service` 는 Docker Compose가 만들어주는 내부 주소입니다. 같은 Docker Compose 안에서는 서비스 이름만으로 서로 통신할 수 있습니다. Docker Compose 설정은 2.7 절에서 다룹니다.

상품 서비스의 재고 감소·복구 API를 호출합니다.

[작성] `adapter/ProductClient.java` - 아래 코드를 해당 파일에 추가하세요.

```
@Component
public class ProductClient { // 상품 서비스 호출 클라이언트

    private final RestClient restClient;

    public ProductClient(RestClient.Builder restClientBuilder) {
        this.restClient = restClientBuilder
            .baseUrl("http://product-service:8082") // 서비스 이름으로 통신 (Docker Compose가 DNS 역할)
            .build();
    }

    public void decreaseQuantity(ProductRequest requestDTO) { // 재고 감소 요청
        restClient.put()
            .uri("/api/products/{productId}/decrease", requestDTO.productId())
            .body(requestDTO)
            .retrieve()
            .toBodilessEntity();
    }

    public void increaseQuantity(ProductRequest requestDTO) { // 재고 복구 요청 (보상 트랜잭션용)
        restClient.put()
            .uri("/api/products/{productId}/increase", requestDTO.productId())
            .body(requestDTO)
            .retrieve()
            .toBodilessEntity();
    }
}
```

배달 서비스의 배달 생성·취소 API를 호출합니다.

[작성] `adapter/DeliveryClient.java` - 아래 코드를 해당 파일에 추가하세요.

```
@Component
public class DeliveryClient { // 배달 서비스 호출 클라이언트

    private final RestClient restClient;

    public DeliveryClient(RestClient.Builder restClientBuilder) {
        this.restClient = restClientBuilder

```

```

        .baseUrl("http://delivery-service:8084") // 서비스 이름으로 통신 (Docker Compose가 DNS 역할)
        .build();
    }

    public void createDelivery(DeliveryRequest requestDTO) { // 배달 생성 요청
        restClient.post()
            .uri("/api/deliveries")
            .body(requestDTO)
            .retrieve()
            .toBodilessEntity();
    }

    public void cancelDelivery(int orderId) { // 배달 취소 요청 (보상 트랜잭션용)
        restClient.put()
            .uri("/api/deliveries/{orderId}", orderId)
            .retrieve()
            .toBodilessEntity();
    }
}

```

2.6.10 OrderService : 보상 트랜잭션의 핵심

이제 이번 챕터의 하이라이트입니다. 플래그 기반 보상 트랜잭션 패턴을 직접 구현합니다. 코드를 읽을 때 `productDecreased` 와 `deliveryCreated` 두 변수를 추적하면서 읽어보세요. 각 단계 성공 시 플래그를 `true` 로 바꾸고, catch 블록에서 플래그를 확인해 보상 여부를 결정합니다.

[작성] `orders/OrderService.java` (createOrder 메서드) – 아래 메서드를 OrderService에 추가하세요.

```

@Transactional
public OrderResponse createOrder(int userId, List<OrderRequest.OrderItemDTO> orderItems, String address) {
    // 보상트랜잭션을 위한 플래그 선언
    boolean productDecreased = false;
    boolean deliveryCreated = false;
    Order createdOrder = null;

    try {
        // 1. 주문 생성 (PENDING)
        createdOrder = orderRepository.save(Order.create(userId));
        final int orderId = createdOrder.getId();
    }
}

```

```

// 2. 상품 재고 차감
orderItems.forEach(item -> productClient.decreaseQuantity(
    new ProductRequest(item.productId(), item.quantity(), item.price()));
productDecreased = true;

// 3. 주문 아이템 저장
List<OrderItem> createdOrderItems = orderItems.stream()
    .map(item -> OrderItem.create(orderId, item.productId(), item.quantity(), item.price()))
    .toList();
orderItemRepository.saveAll(createdOrderItems);

// 4. 배달 생성
deliveryClient.createDelivery(new DeliveryRequest(orderId, address));
deliveryCreated = true;

// 5. 주문 완료
createdOrder.complete();
return OrderResponse.from(createdOrder, createdOrderItems);

} catch (Exception e) {
    // 보상 트랜잭션: 역순으로 되돌리기
    if (deliveryCreated) {
        deliveryClient.cancelDelivery(createdOrder.getId());
    }
    if (productDecreased) {
        orderItems.forEach(item -> productClient.increaseQuantity(
            new ProductRequest(item.productId(), item.quantity(), item.price())
        ));
    }
    throw new Exception500("주문 생성 중 오류가 발생했습니다: " + e.getMessage());
}
}

```

OrderService의 나머지 두 메서드입니다. `findById` 는 주문과 주문 아이템을 조회합니다. `cancelOrder` 는 `createOrder`의 역과정으로, 재고 복구(`increaseQuantity`) → 배달 취소(`cancelDelivery`) → 주문 상태 변경(`cancel()`) 순서로 처리합니다.

[참고] `orders/OrderService.java`

```

// 주문 조회
public OrderResponse findById(int orderId) {

```

```

    Order findOrder = orderRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    List<OrderItem> findOrderItems = orderItemRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문 아이템을 찾을 수 없습니다."));
    return OrderResponse.from(findOrder, findOrderItems);
}

// 주문 취소
@Transactional
public OrderResponse cancelOrder(int orderId) {
    Order findOrder = orderRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    if(findOrder.getStatus() == OrderStatus.CANCELLED) {
        throw new Exception400("주문이 이미 취소되었습니다.");
    }
    List<OrderItem> findOrderItems = orderItemRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문 아이템을 찾을 수 없습니다."));
    // 상품 재고 복구
    findOrderItems.forEach(item -> productClient.increaseQuantity(
        new ProductRequest(item.getProductId(), item.getQuantity(), item.getPrice())
    ));
    // 배달 취소
    deliveryClient.cancelDelivery(orderId);
    // 주문 취소
    findOrder.cancel();
    return OrderResponse.from(findOrder);
}

```

2.7 Docker Compose : 네 개의 서비스를 한 번에 실행하다

코드가 완성되었습니다. 이제 직접 실행하여 보상 트랜잭션이 작동하는 것을 눈으로 확인해 봅니다.

2.7.1 서비스 실행

각 서비스의 Dockerfile은 동일한 구조입니다. 4개 서비스 모두 아래와 같습니다.

[참고] [Dockerfile](#) 스프링 프로젝트를 컨테이너 내에서 실행합니다.

```

FROM gradle:8.14-jdk21 # Gradle + JDK 21 베이스 이미지
WORKDIR /app           # 작업 디렉토리 설정
COPY ..                # 프로젝트 파일 복사
RUN gradle clean bootJar -x test # 테스트 생략, JAR 빌드
RUN cp build/libs/*.jar app.jar # 빌드된 JAR를 app.jar로 복사
ENTRYPOINT ["java", "-jar", "app.jar"] # JAR 실행

```

chap01 디렉토리의 `docker-compose.yml` 로 4개 서비스를 한 번에 실행합니다.

[참고] `docker-compose.yml` 4개 서비스 구조가 동일하므로, 주문 서비스만 예시로 보여줍니다.

```

services:
  order-service:      # 주문 서비스 정의
    build:
      context: ./order # 빌드할 소스 경로
      dockerfile: Dockerfile
    container_name: order-service
    ports:
      - "8081:8081"    # 호스트:컨테이너 포트 매핑
    networks:
      - msa-network    # 같은 네트워크에 연결해야 서비스 이름으로 통신 가능
    depends_on:        # 나머지 세 서비스가 먼저 시작되어야 함
      - user-service
      - product-service
      - delivery-service

# user-service(8083), product-service(8082), delivery-service(8084) 동일 패턴 (Github에서 확인 가능합니다)

networks:
  msa-network:      # 4개 서비스를 하나로 묶는 가상 네트워크

```

`msa-network` 로 묶여 있기 때문에, 컨테이너끼리는 서비스 이름(예: `http://product-service:8082`)으로 통신합니다. `depends_on` 은 주문 서비스가 나머지 세 서비스보다 나중에 시작되도록 보장합니다.

[작성] 프로젝트가 위치한 폴더로 이동 후, 터미널에서 Docker Compose로 4개 서비스를 한 번에 빌드하고 실행합니다.

```

cd chap01
docker compose up

```

실행이 완료되면 각 서비스에 접근할 수 있습니다.

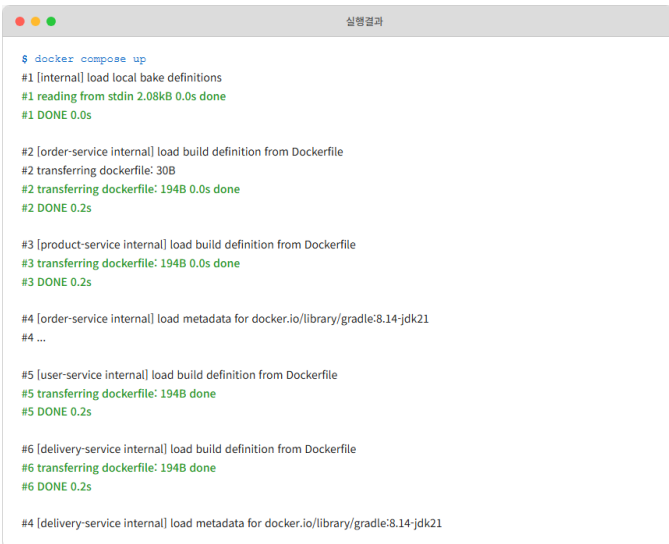
서비스	주소
주문 서비스	http://localhost:8081
상품 서비스	http://localhost:8082
회원 서비스	http://localhost:8083
배달 서비스	http://localhost:8084

2.7.2 사전 준비

실행 시나리오를 따라하려면 두 가지가 필요합니다.

Docker Desktop

Docker가 설치되어 있어야 합니다. <https://www.docker.com/products/docker-desktop/> 에서 설치하세요. Docker Desktop을 실행하고 화면 하단에 “Engine running”이 표시되면 준비 완료입니다. 터미널에서 `docker compose up` 을 실행합니다.



```

$ docker compose up
#1 [internal] load local bake definitions
#1 reading from stdin 2.08kB 0.0s done
#1 DONE 0.0s

#2 [order-service internal] load build definition from Dockerfile
#2 transferring dockerfile: 30B
#2 transferring dockerfile: 194B 0.0s done
#2 DONE 0.2s

#3 [product-service internal] load build definition from Dockerfile
#3 transferring dockerfile: 194B 0.0s done
#3 DONE 0.2s

#4 [order-service internal] load metadata for docker.io/library/gradle:8.14-jdk21
#4 ...

#5 [user-service internal] load build definition from Dockerfile
#5 transferring dockerfile: 194B done
#5 DONE 0.2s

#6 [delivery-service internal] load build definition from Dockerfile
#6 transferring dockerfile: 194B done
#6 DONE 0.2s

#4 [delivery-service internal] load metadata for docker.io/library/gradle:8.14-jdk21

```

그림 0-9: Docker Compose 실행 결과

API 테스트 도구 - Hoppscotch

서비스가 실행되면 API를 호출하여 결과를 확인해야 합니다. 이 책에서는 Hoppscotch(<https://hoppscotch.io/>)를 사용합니다. localhost로 요청을 보내려면 Chrome 웹 스토어에서 **Hoppscotch Browser Extension**을 설치합니다. 설치 후 Hoppscotch 화면 하단의 인터셉터 설정에서 “**Browser Extension**”을 선택하세요.

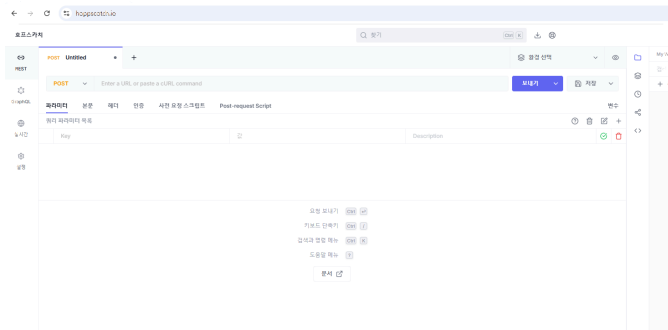


그림 0-10: Hoppscotch 화면

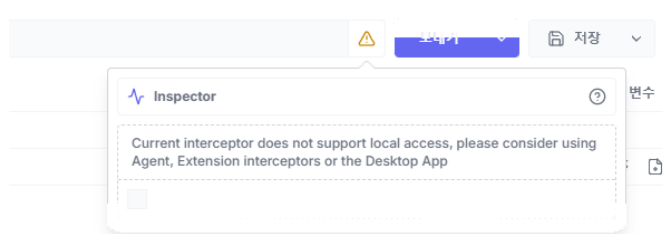


그림 0-11: Browser Extension 인터셉터 설정

2.7.3 시나리오 1: 정상 주문

먼저 로그인하여 JWT 토큰을 받습니다. 이때 콘텐츠 종류(Content-Type)를 `application/json` 으로 설정해야 합니다. 이는 서버에게 “내가 보내는 데이터는 JSON 형식이다”라고 알려주는 것입니다.

POST http://localhost:8083/login

```
{
  "username": "ssar",
  "password": "1234"
}
```

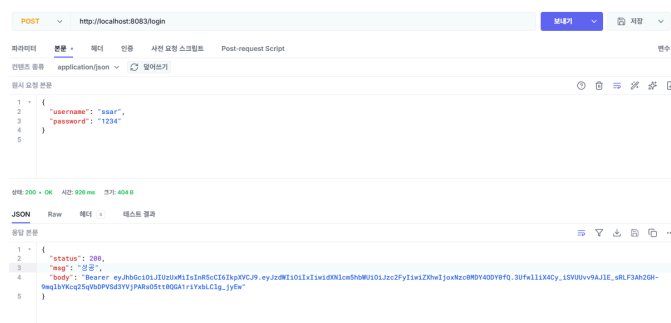


그림 0-12: 로그인 API 호출 결과

응답 바디 데이터에 포함된 JWT 토큰을 확인할 수 있습니다.

받은 토큰을 Hoppscotch의 인증 > 인증 유형(Bearer) 항목의 토큰 필드에 넣습니다.

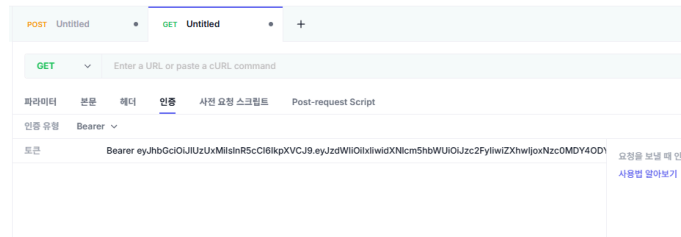


그림 0-13: Bearer 토큰 설정

MacBook Pro(상품 ID 1, 재고 10개)를 1개 주문합니다.

POST <http://localhost:8081/api/orders>

```
{
  "address": "Addr 4",
  "orderItems": [
    {
      "productId": 1,
      "quantity": 1,
      "price": 2500000
    }
  ]
}
```

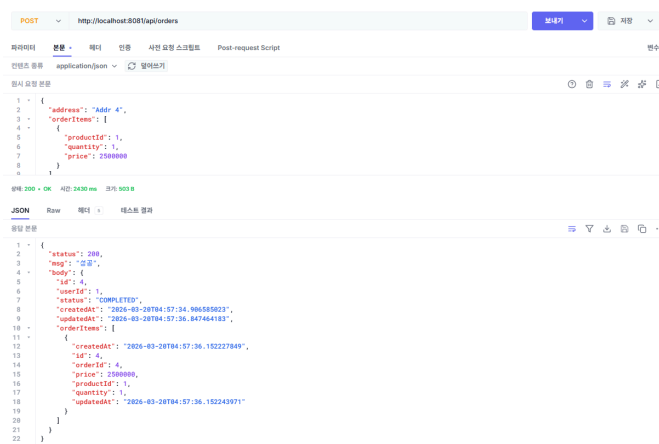


그림 0-14: 주문 생성 API 호출 결과

주문이 성공하면 상품 서비스에서 재고가 10 → 9로 줄어 들고, 배달 서비스에 배달이 생성됩니다.

GET <http://localhost:8082/api/products/1>

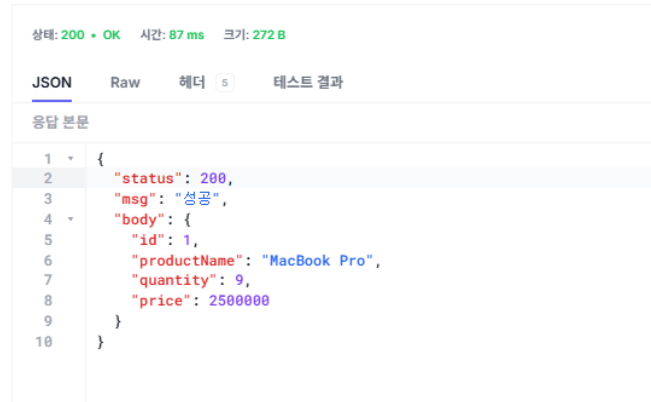


그림 0-15: 재고 감소 확인

GET <http://localhost:8084/api/deliveries/4>

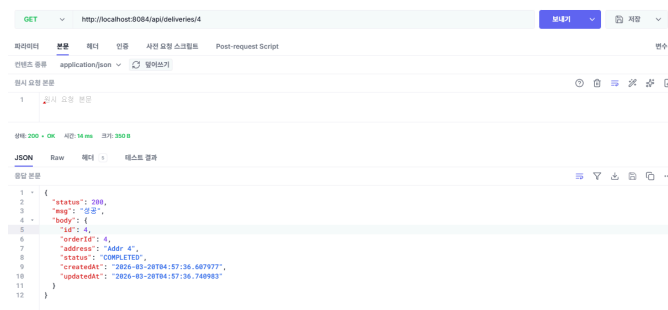


그림 0-16: 배달 생성 확인

2.7.4 시나리오 2: 재고 부족

품질 상품인 iPhone 15(상품 ID 2, 재고 0)를 주문해봅니다. 첫 번째 단계(재고 차감)에서 바로 실패하므로 보상할 작업이 없어 즉시 에러가 반환됩니다.

POST <http://localhost:8081/api/orders>

```

{
  "address": "Addr 4",
  "orderItems": [
    {
      "productId": 2,
  
```

```

    "quantity": 1,
    "price": 1300000
  }
]
}

```



그림 0-17: 재고 부족 시 에러 응답

2.7.5 시나리오 3: 주소 누락

이번에는 주소를 빈 문자열로 보내봅니다. 주문 자체는 재고 차감까지 진행되지만, 배달 서비스에서 주소가 없으므로 실패합니다. 이때 보상 트랜잭션이 작동하여 차감된 재고가 복구됩니다.

POST <http://localhost:8081/api/orders>

```

{
  "address": "",
  "orderItems": [
    {
      "productId": 1,
      "quantity": 1,
      "price": 2500000
    }
  ]
}

```

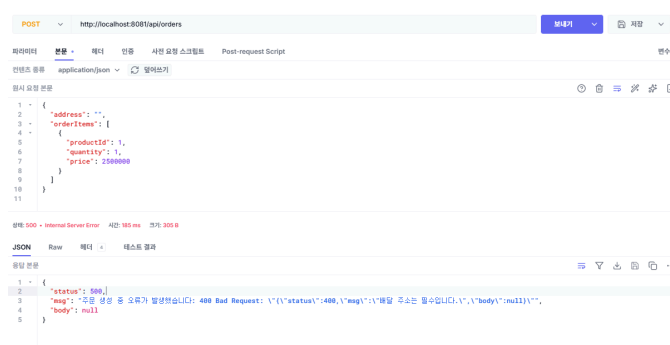


그림 0-18: 주소 누락 시 에러 응답

그리고 재고가 원복되었는지 확인합니다.

GET http://localhost:8082/api/products/1

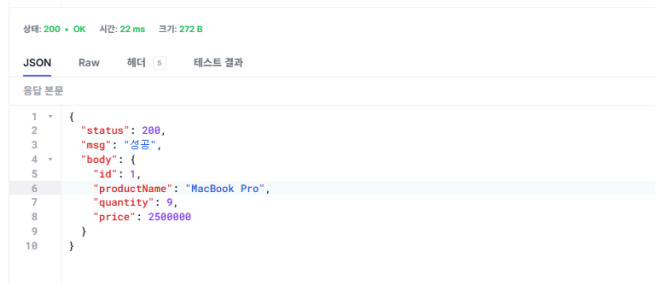


그림 0-19: 재고 원복 확인

테스트가 끝났으면 실행 중인 컨테이너를 정리합니다.

docker compose down

이 명령어를 실행하면 docker compose up으로 띄운 모든 컨테이너가 종료되고 제거됩니다.

정리

이번 챕터에서 만든 것을 정리합니다.

- 4개의 Spring Boot 서비스를 독립적으로 실행하고 REST로 연결했습니다.
- JWT 인증 필터를 구현하여 모든 서비스가 독립적으로 토큰을 검증합니다.
- 주문 서비스에서 RestClient 인터셉터로 Authorization 헤더를 하위 서비스에 자동 전달합니다.
- 플래그 기반 보상 트랜잭션으로 주문 실패 시 재고와 배달을 원상복구합니다.

보상 트랜잭션 덕분에 데이터 일관성을 유지할 수 있었습니다. 하지만 이 구조에는 눈에 잘 보이지 않는 문제가 있습니다.

이 구조의 한계: - 모든 서비스 호출이 동기적입니다. 상품 서비스가 1초 걸리면 주문 서비스 전체가 1초를 기다립니다. - 보상 트랜잭션 코드가 비즈니스 로직과 섞여 복잡합니다. 서비스가 늘어날수록 try-catch 중첩이 깊어집니다. - 지금은 로컬에서만 실행됩니다. 실제 운영 서버에 올리려면 배포 방법을 별도로 고민해야 합니다.

코드 구조가 복잡해지는 것도 문제지만, 더 근본적인 문제는 운영 가능한 형태로 만들어야 한다는 것입니다. 다음 챕터에서는 코드 구조를 Clean Architecture로 개선하고, Kubernetes로 배포하는 방법을 배웁니다.

챕터 3. 클린 아키텍처와 Kubernetes

운영 환경

chap02 · 실행 환경: 컨테이너 · UseCase 인터페이스 · MySQL · K8s 이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/chap02> 에서 확인할 수 있습니다. 이번 챕터는 직접 코드를 작성하지 않습니다. 챕터 2 프로젝트를 클린 아키텍처로 재구성한 결과를 살펴보고, Kubernetes 배포를 실습합니다.

학습 목표

- 컨트롤러가 서비스 구현체 대신 UseCase 인터페이스에 의존하도록 구조를 개선한다.
- MySQL을 연결하고 개발/운영 프로파일을 분리한다.
- Docker 이미지를 빌드하고 Kubernetes에 배포한다.
- ConfigMap과 Secret으로 환경 변수를 주입하는 방법을 이해한다.

3.1 챕터 2가 남긴 두 가지 숙제

챕터 2에서 4개 서비스가 REST로 통신하는 시스템을 만들었습니다. 동작은 합니다. 하지만 실제 서비스로 운영하기에는 두 가지 문제가 있습니다.

첫 번째 문제: 코드 구조. OrderController는 OrderService에 직접 의존합니다. 나중에 OrderService를 교체하거나 가짜 데이터(Mock)로 바꿔 테스트하려면 Controller 코드도 건드려야 합니다. 서비스가 많아질수록 이 결합이 코드 전체를 딱딱하게 만듭니다.

두 번째 문제: 운영 환경. 챕터 2에서는 H2 인메모리 DB로 로컬에서만 실행했습니다. 실제 운영에서는 MySQL 같은 영구 저장소를 사용하고, DB 접속 정보나 비밀번호 같은 설정값을 코드가 아닌 외부에서 주입받아야 합니다.

이번 챕터는 이 두 숙제를 동시에 해결하는 이야기입니다. 코드 구조는 클린 아키텍처(UseCase 인터페이스)로, 운영 환경은 Kubernetes로 해결합니다. 챕터 2의 Docker Compose는 서비스를 한 번에 띄우기엔 편하지만, 서비스가 죽으면 직접 다시 띄워야 하고 특정 서비스만 늘리기도 어렵습니다. Kubernetes는 이런 부분을 자동으로 처리합니다.

클린 아키텍처(**Clean Architecture**): 비즈니스 규칙을 중심에 두고, 외부 기술(DB, 웹 프레임워크 등)이 안쪽 규칙에 의존하도록 계층을 나누는 소프트웨어 설계 방식입니다.

3.2 UseCase : 왜 인터페이스인가

클린 아키텍처에서 컨트롤러는 구현체가 아닌 **UseCase** 인터페이스에 의존합니다. 구현체가 바뀌어도 컨트롤러는 수정할 필요가 없고, 테스트할 때도 가짜 구현체를 쉽게 넣을 수 있습니다.

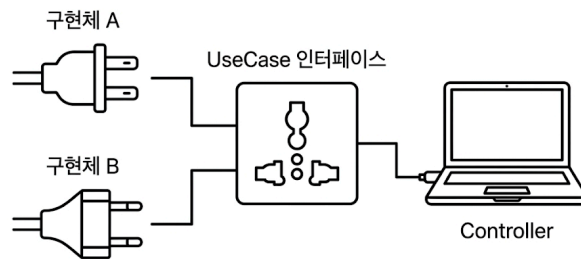


그림 0-1: UseCase 인터페이스 — 전원 어댑터 비유

코드로 옮기면 이렇게 됩니다.

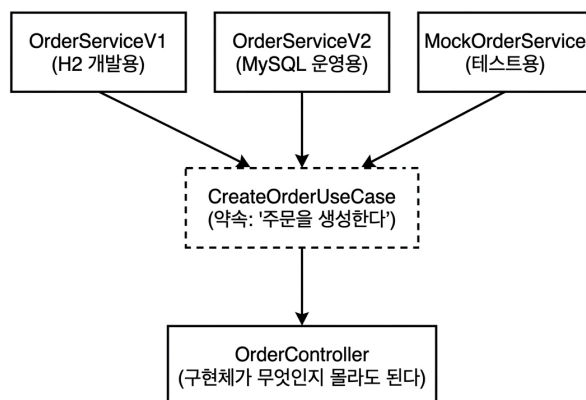


그림 0-2: 챕터 2 vs 챕터 3 의존 구조 비교

[2장 구조 - 직접 의존]

OrderController —————> **OrderService** (구현체)

문제: **OrderService** 구현체가 변경되면 **OrderController**도 영향을 받을 수 있다.

[3장 구조 - 인터페이스 의존]

OrderController —————> **CreateOrderUseCase** (인터페이스)

▲

OrderService (구현체)

장점: **OrderController**는 인터페이스만 알면 된다.

구현체를 교체해도 컨트롤러 코드는 변경이 없다.

“무엇을 할 것인가”(UseCase 인터페이스)와 “어떻게 할 것인가”(Service 구현체)를 분리하는 것이 핵심입니다.

3.2.1 챕터 2 vs 챕터 3 코드 비교

코드로 보면 차이가 더 명확합니다. 컨트롤러가 의존하는 대상이 구현체에서 인터페이스로 바뀝니다.

```
// 2장: 구현체에 직접 의존
private final OrderService orderService;

// 3장: UseCase 인터페이스에 의존
private final CreateOrderUseCase createOrderUseCase;
private final GetOrderUseCase getOrderUseCase;
private final CancelOrderUseCase cancelOrderUseCase;
```

3.3 UseCase 인터페이스 도입

UseCase 인터페이스가 왜 필요한지 이해했으니, 이제 실제 코드에 적용해보겠습니다. 패키지 구조부터 바꾸겠습니다. 챕터 2의 단순 레이어드 구조에서 클린 아키텍처 구조로 전환합니다.

3.3.1 패키지 구조 변경

```

chap02/order/src/main/java/com/metacoding/order/
├── domain/                # JPA 엔티티
│   ├── Order.java        # [참고]
│   ├── OrderItem.java    # [참고]
│   └── OrderStatus.java   # [참고]
├── repository/           # Spring Data JPA
│   └── OrderRepository.java # [참고]
├── usecase/              # UseCase 인터페이스 + 서비스 구현체
│   ├── CreateOrderUseCase.java # [참고]
│   ├── GetOrderUseCase.java    # [참고]
│   ├── CancelOrderUseCase.java # [참고]
│   └── OrderService.java       # [참고]
├── web/                  # 컨트롤러 + DTO
│   ├── OrderController.java   # [참고]
│   └── dto/
│       ├── OrderRequest.java  # [참고]
│       └── OrderResponse.java  # [참고]
├── adapter/              # 외부 서비스 클라이언트 (order 전용)
│   ├── ProductClient.java     # [참고] 2장과 동일
│   ├── DeliveryClient.java    # [참고] 2장과 동일
│   └── dto/                   # 어댑터용 DTO
│       ├── ProductRequest.java # [참고]
│       └── DeliveryRequest.java # [참고]
└── core/                 # JWT, 예외처리 (2장과 동일)

```

user/product/delivery도 동일한 구조이며, adapter/ 패키지만 order 전용입니다.

참고: 이 책에서는 클린 아키텍처의 핵심인 UseCase 인터페이스를 통한 의존성 역전에 집중합니다. 완전한 아키텍처보다는 실습에 필요한 개념만 적용합니다.

3.3.2 UseCase 인터페이스 정의

주문 생성·조회·취소를 각각 별도 인터페이스로 표현합니다. 인터페이스 하나가 하나의 행위(Use Case)를 표현하도록 합니다.

[참고] `usecase/CreateOrderUseCase.java` , `GetOrderUseCase.java` , `CancelOrderUseCase.java` – 동작 이해용입니다.

```
public interface CreateOrderUseCase {
    OrderResponse createOrder(int userId, List<OrderRequest.OrderItemDTO> orderItems, String address);
}

public interface GetOrderUseCase {
    OrderResponse findById(int orderId);
}

public interface CancelOrderUseCase {
    OrderResponse cancelOrder(int orderId);
}
```

3.3.3 엔티티의 비즈니스 로직

“주문이 취소 가능한가?” 같은 비즈니스 규칙은 서비스가 아닌 엔티티에 둡니다. 엔티티 메서드로 캡슐화하면 어디서 호출하던 동일한 규칙이 적용됩니다.

[참고] `domain/Order.java` (`validateCancelable` 추가) – 동작 이해용입니다.

```
public class Order {
    // 2장 Order.java 참조 – 필드 및 create(), complete(), cancel() 동일

    // 비즈니스 규칙을 엔티티에 위임 (3장에서 추가)
    public void validateCancelable() {
        if (this.status == OrderStatus.CANCELLED) {
            throw new Exception400("주문이 이미 취소되었습니다.");
        }
    }
}
```

3.3.4 OrderService : 인터페이스 구현

OrderService는 세 UseCase 인터페이스를 구현하고, 내부에서 도메인 객체의 비즈니스 메서드를 호출합니다. 이 구조를 통해 서비스는 흐름 조율에만 집중하고, 실제 비즈니스 규칙은 도메인이 담당하도록 책임을 분리합니다.

보상 트랜잭션 로직은 챕터 2와 동일합니다. 달라진 점은 다음과 같습니다.

1. **UseCase** 인터페이스 구현 – 서비스가 직접 메서드를 노출하지 않고, `CreateOrderUseCase` 등 인터페이스를 구현합니다.
2. 비즈니스 규칙을 엔티티에 위임 – 챕터 2에서 서비스의 `if` 문으로 처리하던 검증 (`validateQuantity`, `validatePrice` 등)을 도메인 객체의 메서드로 이동합니다.

[참고] [usecase/OrderService.java](#)

```
@Service
@Transactional(readOnly = true)           // 1. 클래스 레벨 읽기 전용 트랜잭션
public class OrderService implements CreateOrderUseCase, GetOrderUseCase, CancelOrderUseCase {
    // 2. UseCase 인터페이스를 구현

    @Override
    @Transactional                       // 쓰기 메서드만 오버라이드
    public OrderResponse cancelOrder(int orderId) {
        // ...

        findOrder.validateCancelable();    // 3. 검증 로직을 엔티티에 위임
        findOrder.cancel();
        // ...
    }
}
```

전체 코드는 GitHub에서 확인하세요.

3.3.5 OrderController 수정

구현체가 아닌 인터페이스를 주입받도록 컨트롤러를 수정합니다. 앞으로 `OrderService`를 다른 구현체로 바꿔도 이 컨트롤러는 전혀 수정하지 않아도 됩니다. API는 챕터 2와 동일합니다(POST 생성, GET 조회, PUT 취소).

[참고] [web/OrderController.java](#)

```
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final CreateOrderUseCase createOrderUseCase; // 구현체가 아닌 인터페이스 주입
    private final GetOrderUseCase getOrderUseCase;
    private final CancelOrderUseCase cancelOrderUseCase;
```

```

@PostMapping
public ResponseEntity<?> createOrder(...) {
    return Resp.ok(createOrderUseCase.createOrder(...)); // 인터페이스 메서드 호출
}

// GET /{orderId} - 주문 조회
// PUT /{orderId} - 주문 취소
}

```

전체 코드는 GitHub에서 확인하세요.

3.3.6 나머지 서비스의 UseCase 적용

order-service와 동일한 패턴으로 나머지 세 서비스도 UseCase 인터페이스를 도입합니다.

서비스	UseCase 인터페이스	엔티티 검증 메서드
product	GetProductUseCase, GetAllProductsUseCase, DecreaseQuantityUseCase, IncreaseQuantityUseCase	validateQuantity() , validatePrice()
delivery	SaveDeliveryUseCase, GetDeliveryUseCase, CancelDeliveryUseCase	validateAddress() , validateCancelable()
user	LoginUseCase, GetUserUseCase, GetAllUsersUseCase	validatePassword()

챕터 2에서 Service의 `if` 문으로 처리하던 검증 로직이 엔티티 메서드로 이동합니다.

전체 코드는 GitHub에서 확인하세요.

첫 번째 숙제(코드 구조)를 해결했습니다. 이제 두 번째 숙제, 운영 환경 배포를 해결할 차례입니다.

3.4 MySQL : 운영 데이터베이스 연결과 프로파일 분리

운영 환경에서는 서비스가 재시작되더라도 데이터가 유지되어야 합니다. 실제 사용자 데이터는 서버가 꺼져도 남아있어야 합니다. H2는 메모리에만 저장되므로 운영에는 MySQL 같은 외부 데이터베이스를 사용합니다.

`application.properties` 에서 개발(H2)과 운영(MySQL) 환경을 분리합니다.

```
src/main/resources/
├─ application.properties    # 공통 설정, active profile 지정
├─ application-dev.properties # 개발: H2 설정
└─ application-prod.properties # 운영: MySQL, 환경변수 참조
```

운영 프로파일은 환경변수를 참조합니다. `${DB_URL}` 처럼 플레이스홀더를 사용하면 코드를 변경하지 않고 Kubernetes ConfigMap/Secret에서 환경 변수 값을 주입받을 수 있습니다.

[참고] `application-prod.properties` – 동작 이해용입니다.

```
# Database (환경변수에서 읽음)
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=${DDL_AUTO:validate}

# 생략...
```

MySQL을 사용하려면 `build.gradle`에 드라이버 의존성을 추가해야 합니다.

[참고] `build.gradle` (변경 부분) – 동작 이해용입니다.

```
dependencies {
    // 생략...

    runtimeOnly 'com.mysql:mysql-connector-j' // 신규 추가

    // 생략...
}
```

3.5 Docker : 이미지 빌드와 인프라 구성

이번 챕터부터는 Docker 이미지를 Minikube 위에서 실행합니다. Minikube가 설치되어 있지 않으면 공식 사이트(<https://minikube.sigs.k8s.io/>)에서 설치 후 `minikube start` 명령어로 클러스터를 시작합니다.

3.5.1 Nginx : API Gateway 라우팅

챕터 2에서는 각 서비스 포트(8081~8084)로 직접 접근했습니다. 서비스가 늘어나면 클라이언트가 포트를 전부 알아야 하므로, 하나의 진입점으로 통합합니다. Nginx를 API Gateway로 두면, 클라이언트는 하나의 진입점(80번 포트)으로 요청하고 URL 경로에 따라 적절한 서비스로 라우팅됩니다.

```
gateway/
├── Dockerfile    # Nginx 이미지 빌드 [참고]
└── nginx.conf    # URL 경로별 라우팅 설정 [참고]
```

Dockerfile은 Nginx를 설치하고, 우리가 작성한 설정 파일을 넣어주는 역할입니다.

[참고] [gateway/Dockerfile](#)

```
FROM nginx:alpine          # 경량 Nginx 이미지
COPY nginx.conf /etc/nginx/nginx.conf  # 라우팅 설정 파일 복사
EXPOSE 80                  # 게이트웨이 포트
CMD ["nginx", "-g", "daemon off;"]    # 포그라운드 실행
```

nginx.conf는 어떤 URL이 들어오면 어느 서비스로 보낼지를 정하는 설정 파일입니다.

[참고] [gateway/nginx.conf](#)

```
events {}

http {
    # 각 서비스를 upstream 블록으로 등록
    upstream user-service {
        server user-service:8083;    # K8s 내부 DNS로 서비스 접근
    }

    # product-service(8082), order-service(8081), delivery-service(8084)도 동일 패턴

    server {
        listen 80;                  # 게이트웨이 진입점
        server_name localhost;

        # URL 경로별로 요청을 해당 서비스로 분기
        location /login {
            proxy_pass http://user-service;
        }

        location /api/users {
```

```

    proxy_pass http://user-service;
  }
  # /api/products → product-service, /api/orders → order-service,
  # /api/deliveries → delivery-service도 동일 패턴
}
}

```

`upstream` 블록에 4개 서비스를 등록하고, `location` 으로 URL 경로를 분기합니다. `user-service`, `order-service` 같은 이름은 Kubernetes가 내부 DNS로 자동 해석하므로, IP 주소를 직접 지정할 필요가 없습니다.

전체 코드는 GitHub에서 확인하세요.

3.5.2 MySQL : 데이터베이스 인프라

모든 서비스가 동일한 MySQL 인스턴스(`db-service:3306`)의 `metadb` 데이터베이스를 공유합니다. 서비스별로 테이블이 분리되어 있으나, 물리적으로는 단일 DB 인스턴스입니다.

참고: 실제 MSA에서는 서비스마다 독립된 DB를 둡니다. 이 책에서는 학습 편의를 위해 하나의 MySQL을 공유합니다. Saga 패턴을 익히는 데는 차이가 없으니 DB 구성보다 흐름에 집중해 주세요.

DB 컨테이너는 `db/` 디렉토리의 `Dockerfile`과 `init.sql`로 구성됩니다.

```

db/
├── Dockerfile # [참고] MySQL 이미지 빌드
└── init.sql   # [참고] 테이블 생성 + 더미 데이터

```

[참고] `db/Dockerfile`

```

FROM mysql                # MySQL 공식 이미지
COPY init.sql /docker-entrypoint-initdb.d # 컨테이너 최초 시작 시 자동 실행
ENV MYSQL_ROOT_PASSWORD=root1234 # root 비밀번호
ENV MYSQL_DATABASE=metadb      # 기본 데이터베이스 생성
CMD ["--character-set-server=utf8mb4", "--collation-server=utf8mb4_unicode_ci"]

```

`init.sql` 은 서비스별 테이블 생성과 더미 데이터 삽입을 담당합니다. 챕터 2의 H2 `data.sql` 은 INSERT 만 있었지만, MySQL은 자동으로 테이블을 만들어주지 않으므로 CREATE TABLE도 포함합니다.

[참고] `db/init.sql`

```
-- 테이블 생성 (4개 서비스의 5개 테이블)
CREATE TABLE user_tb ( id INT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50) UNIQUE NOT NULL, ... );
CREATE TABLE product_tb ( id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(50), ... );
CREATE TABLE order_tb ( id INT AUTO_INCREMENT PRIMARY KEY, user_id INT, status VARCHAR(50), ... );
CREATE TABLE order_item_tb ( id INT AUTO_INCREMENT PRIMARY KEY, order_id INT, product_id INT, ... );
CREATE TABLE delivery_tb ( id INT AUTO_INCREMENT PRIMARY KEY, order_id INT, address VARCHAR(50), ... );

-- 더미 데이터 (2장과 동일)
```

3.6 Kubernetes : YAML로 선언하는 배포

3.6.1 매니페스트 구조 설계

Kubernetes는 YAML 파일로 원하는 상태를 선언합니다. “이 서비스는 이렇게 실행되어야 한다” 고 파일에 적어두면, K8s가 그 상태를 유지합니다.

ConfigMap 과 **Secret** 은 환경변수를 저장합니다. **Deployment** 는 컨테이너를 어떻게 실행할지 정의합니다. **Service** 는 **Pod** 에 고정 주소(DNS)를 부여하여 클러스터 내부에서 접근할 수 있게 합니다. 외부 요청은 **Ingress** 가 받아 적절한 Service로 전달합니다.



그림 0-4: Kubernetes 리소스 관계

각 서비스마다 4가지 리소스가 필요합니다.

```
chap02/k8s/
├── db/
│   ├── db-configmap.yml # [참고] DB 연결 설정
│   ├── db-deployment.yml # [참고] MySQL Pod
│   ├── db-secret.yml # [참고] DB 비밀번호
│   └── db-service.yml # [참고] MySQL 서비스 (ClusterIP)
```

```

└─ order/
  └─ order-configmap.yml # [참고] 일반 환경 변수
  └─ order-deploy.yml   # [참고] order-service Pod
  └─ order-secret.yml    # [참고] 민감 정보
  └─ order-service.yml   # [참고] 서비스 노출
└─ gateway/
  └─ gateway-deploy.yml # [참고] gateway Pod
  └─ gateway-service.yml # [참고] 서비스 노출
  └─ gateway-ingress.yml # [참고] 외부 요청 라우팅
└─ product/ user/ delivery/ # order와 동일 패턴

```

각 K8s 리소스의 역할을 정리하면 다음과 같습니다.

리소스	파일	역할	주요 값
ConfigMap	order-configmap.yml	일반 환경변수	<code>DB_URL</code> , <code>DB_DRIVER</code>
Secret	order-secret.yml	민감한 환경변수	<code>DB_USERNAME</code> , <code>DB_PASSWORD</code>
Deployment	order-deploy.yml	Pod 실행 정의	이미지, 포트, <code>env</code> 로 <code>SPRING_PROFILES_ACTIVE</code> 직접 설정, <code>envFrom</code> 으로 ConfigMap·Secret 주입
Service	order-service.yml	클러스터 내부 통신	Pod에 고정 DNS 부여 (<code>order-service:8081</code>)
Ingress	gateway-ingress.yml	외부 요청 라우팅	모든 외부 요청을 <code>gateway-service:80</code> 으 로 전달

각 리소스가 실제로 어떻게 생겼는지, order 서비스를 예시로 하나씩 살펴보겠습니다.

ConfigMap : 일반 환경변수 주입

[참고] [k8s/order/order-configmap.yml](#)


```

apiVersion: v1
kind: ConfigMap
metadata:
  name: order-configmap
  namespace: metacoding
data:
  # 키-값 쌍으로 환경변수 저장
  DB_URL: jdbc:mysql://db-service:3306/metadb?useSSL=false&serverTimezone=UTC&useLegacyDatetimeC
ode=false&allowPublicKeyRetrieval=true
  DB_DRIVER: com.mysql.cj.jdbc.Driver

```

ConfigMap은 애플리케이션이 필요로 하는 일반 설정값을 외부에서 주입하는 역할을 합니다. 코드를 수정하지 않고도 DB 주소나 드라이버 같은 설정을 바꿀 수 있습니다.

Secret : 민감한 정보 관리

[참고] [k8s/order/order-secret.yml](#)

```

apiVersion: v1
kind: Secret
metadata:
  name: order-secret
  namespace: metacoding
type: Opaque
stringData:
  # 평문으로 작성하면 K8s가 자동으로 Base64 인코딩
  DB_USERNAME: metacoding
  DB_PASSWORD: metacoding1234

```

Secret은 비밀번호, 인증 정보 같은 민감한 값을 안전하게 관리하는 역할을 합니다. ConfigMap과 구조는 비슷하지만, 노출되면 안 되는 값은 반드시 Secret으로 분리합니다.

Deployment : Pod 실행 정의

[참고] [k8s/order/order-deploy.yml](#)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-deploy
  namespace: metacoding
spec:
  replicas: 1
  selector:
    # Pod를 몇 개 띄울지

```

```

matchLabels:
  app: order
template:
  metadata:
    labels:
      app: order      # Service가 이 라벨로 Pod를 찾을
spec:
  containers:
    - name: order-server
      image: metacoding/order:1
      imagePullPolicy: Never
      ports:
        - containerPort: 8081
      env:              # 개별 환경변수 직접 설정
        - name: SPRING_PROFILES_ACTIVE
          value: "prod"
      envFrom:          # ConfigMap-Secret의 모든 값을 한꺼번에 주입
        - configMapRef:
            name: order-configmap
        - secretRef:
            name: order-secret

```

Deployment는 컨테이너를 어떻게 실행할지 정의하고, **Pod**의 생명주기를 관리하는 역할을 합니다. Pod가 죽으면 자동으로 다시 띄워주고, **replicas**를 늘리면 같은 Pod를 여러 개 실행할 수도 있습니다.

Service : 클러스터 내부 통신

[참고] [k8s/order/order-service.yml](#)

```

apiVersion: v1
kind: Service
metadata:
  name: order-service
  namespace: metacoding
spec:
  type: ClusterIP      # 클러스터 내부에서만 통신
  selector:
    app: order          # 이 라벨을 가진 Pod에 트래픽 전달
  ports:

```

```
- port: 8081      # 다른 서비스가 접근하는 포트
  targetPort: 8081 # Pod 내부의 실제 포트
```

Service는 **Pod**에 고정 주소(**DNS**)를 부여하는 역할을 합니다. Pod는 재시작될 때마다 IP가 바뀌지만, Service 덕분에 다른 서비스들은 항상 `order-service:8081` 이라는 이름으로 접근할 수 있습니다.

Ingress : 외부 요청 라우팅

[참고] `k8s/gateway/gateway-ingress.yml`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: gateway-ingress
  namespace: metacoding
spec:
  rules:
    - http:      # host를 생략하면 모든 도메인의 요청을 받음
      paths:
        - path: /      # 모든 경로를 매칭
          pathType: Prefix
          backend:
            service:
              name: gateway-service
              port:
                number: 80
```

Ingress는 클러스터 외부에서 들어오는 요청을 내부 **Service**로 연결하는 출입구 역할을 합니다. Service가 클러스터 내부의 주소록이라면, Ingress는 외부 세계와 클러스터를 연결하는 정문입니다. 나머지 서비스(product, user, delivery)도 동일한 패턴입니다.

전체 YAML은 GitHub에서 확인하세요.

3.7 Minikube : 실행 및 결과 확인

3.7.1 Minikube 시작

Minikube는 로컬 PC에 가벼운 Kubernetes 클러스터를 만들어주는 도구입니다. Docker Desktop이 실행 중인 상태에서 아래 명령을 입력하면 클러스터가 생성됩니다.

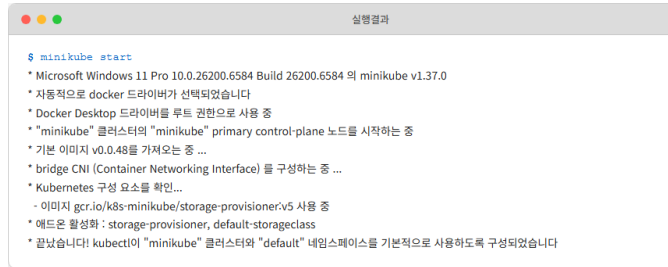
minikube start

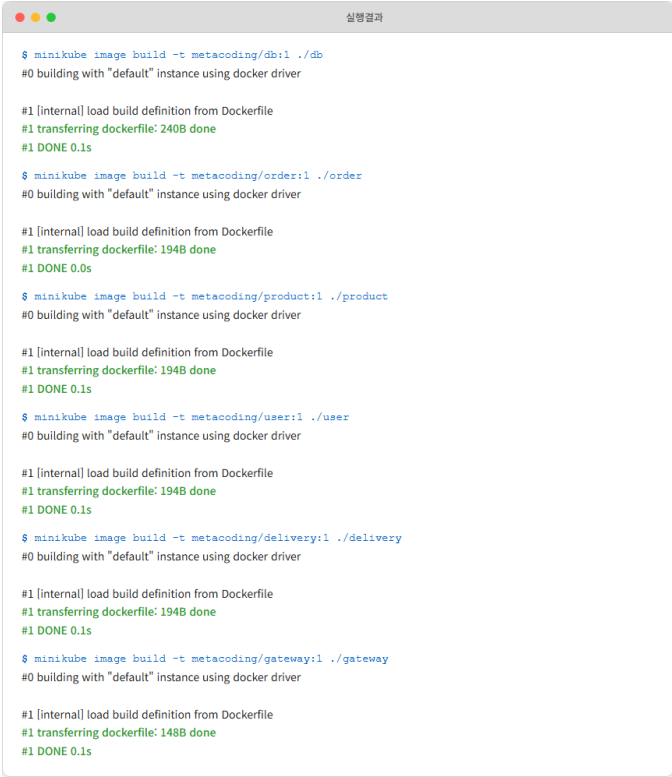
그림 0-6: Minikube 시작

처음 실행하면 필요한 이미지를 다운로드하므로 몇 분 정도 걸릴 수 있습니다.

3.7.2 이미지 빌드

minikube image build 는 Minikube 내부에 직접 이미지를 빌드합니다.

```
minikube image build -t metacoding/db:1 ./db
minikube image build -t metacoding/order:1 ./order
minikube image build -t metacoding/product:1 ./product
minikube image build -t metacoding/user:1 ./user
minikube image build -t metacoding/delivery:1 ./delivery
minikube image build -t metacoding/gateway:1 ./gateway
```



```

$ minikube image build -t metacoding/db:1 ./db
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 240B done
#1 DONE 0.1s

$ minikube image build -t metacoding/order:1 ./order
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 194B done
#1 DONE 0.0s

$ minikube image build -t metacoding/product:1 ./product
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 194B done
#1 DONE 0.1s

$ minikube image build -t metacoding/user:1 ./user
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 194B done
#1 DONE 0.1s

$ minikube image build -t metacoding/delivery:1 ./delivery
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 194B done
#1 DONE 0.1s

$ minikube image build -t metacoding/gateway:1 ./gateway
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 148B done
#1 DONE 0.1s

```

그림 0-7: 이미지 빌드 결과

3.7.3 배포 순서

네임스페이스를 먼저 생성하고, DB가 준비된 뒤에 나머지 서비스를 배포합니다.

1. 네임스페이스 생성 (최초 1회)

kubectl create namespace metacoding

2. DB 관련 리소스 먼저 배포

kubectl apply -f k8s/db

3. 각 서비스 배포

kubectl apply -f k8s/order

kubectl apply -f k8s/product

kubectl apply -f k8s/user

kubectl apply -f k8s/delivery

kubectl apply -f k8s/gateway

4. Ingress 활성화 (Minikube에서는 애드온 활성화 필요)

minikube addons enable ingress

```

$ kubectl create namespace metacoding
namespace/metacoding created

$ kubectl apply -f k8s/db
configmap/db-configmap created
deployment.apps/db-deploy created
secret/db-secret created
service/db-service created

$ kubectl apply -f k8s/order
configmap/order-configmap created
deployment.apps/order-deploy created
secret/order-secret created
service/order-service created

$ kubectl apply -f k8s/product
configmap/product-configmap created
deployment.apps/product-deploy created
secret/product-secret created
service/product-service created

$ kubectl apply -f k8s/user
configmap/user-configmap created
deployment.apps/user-deploy created
secret/user-secret created
service/user-service created

$ kubectl apply -f k8s/delivery
configmap/delivery-configmap created
deployment.apps/delivery-deploy created
secret/delivery-secret created
service/delivery-service created

$ kubectl apply -f k8s/gateway
deployment.apps/gateway-deploy created
service/gateway-service created
ingress.networking.k8s.io/gateway-ingress created

```

그림 0-8: 네임스페이스 생성 및 배포

3.7.4 배포 상태 확인

kubectl get pods -n metacoding

```

$ kubectl get pods -n metacoding
NAME                                READY  STATUS   RESTARTS   AGE
db-deploy-5c94cbf5f-xmlf2           1/1    Running  0           4m11s
delivery-deploy-56cfc89bdd-5x49x    1/1    Running  1 (3m13s ago) 4m11s
gateway-deploy-5d4c49c94c-k8qrk     1/1    Running  0           4m11s
order-deploy-665d89c8fc-glnmc       1/1    Running  1 (2m50s ago) 4m11s
product-deploy-6f5745c6b9-n8jn9     1/1    Running  1 (2m50s ago) 4m11s
user-deploy-5d786978bd-nc9wr        1/1    Running  1 (2m51s ago) 4m10s

```

그림 0-9: Pod 상태 확인

모든 Pod가 **Running** 상태가 되면 배포 완료입니다.

3.7.5 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel` 을 실행합니다.

`minikube tunnel`

터널이 실행되면 `http://127.0.0.1:80` 로 `gateway-service`에 접속할 수 있습니다. `POST http://127.0.0.1:80/login` 으로 로그인하여 토큰을 받습니다. 이후 과정은 챕터 2와 동일하게 주문을 생성합니다.

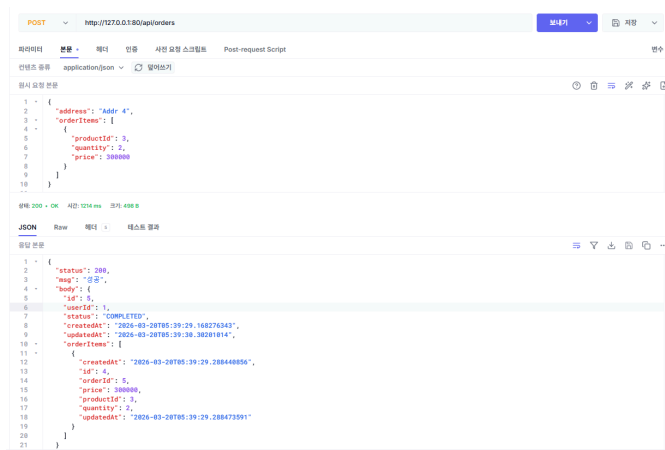


그림 0-10: 주문 결과 확인

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

`kubectl delete all --all -n metacoding`

정리

이번 챕터에서 두 가지 숙제를 해결했습니다.

- **Clean Architecture:** 컨트롤러가 UseCase 인터페이스에만 의존하도록 구조를 분리했습니다. 구현체를 교체해도 컨트롤러는 변경이 없습니다.
- **MySQL + 프로파일 분리:** 개발 환경은 H2, 운영 환경은 MySQL을 사용합니다. 환경변수로 설정을 주입하여 코드 변경 없이 환경을 전환합니다.
- **Nginx API Gateway:** 클라이언트는 게이트웨이 하나로 요청하고, URL 경로에 따라 각 서비스로 라우팅됩니다.

- **Kubernetes 배포:** ConfigMap/Secret으로 환경변수를 안전하게 주입하고, Deployment/Service로 Pod를 관리합니다.

코드 구조도 좋아졌고, 운영 배포도 됩니다. 그런데 아직 해결하지 못한 문제가 있습니다. `product-service`에 장애가 생기면 `order-service`도 그대로 멈춥니다. 동기 호출의 한계입니다. 한 서비스의 문제가 연쇄적으로 다른 서비스에 영향을 줍니다.

다음 챕터에서는 `Kafka`로 서비스 간 통신을 비동기로 전환합니다. 서비스끼리 직접 연결하는 대신 메시지 큐를 사이에 둡니다. 한 서비스가 느려지거나 잠깐 멈춰도 전체 시스템은 계속 동작합니다.

챕터 4. 비동기 MSA - Kafka로 서비스를 분리하다

chap03 · 실행 환경: 컨테이너 · Kafka · Orchestration Saga · orchestrator 신규 이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/chap03> 에서 확인할 수 있습니다.

학습 목표

- 동기 REST 호출의 한계를 이해하고 비동기 메시지 방식의 장점을 설명한다.
- Kafka의 토픽, 프로듀서, 컨슈머, 컨슈머 그룹 개념을 이해한다.
- Orchestration Saga 패턴을 Kafka로 구현한다.
- order-service의 동기 호출을 Kafka 이벤트 발행으로 교체한다.
- orchestrator가 워크플로우 상태를 추적하고 실패 시 자동 롤백하는 방법을 이해한다.

4.1 한 서비스의 장애가 전체를 멈추다

챕터 3을 마치면서 이런 질문을 남겼습니다. “**product-service**가 잠깐 다운되면 어떻게 되나?”

답은 단순합니다. order-service도 주문을 처리할 수 없게 됩니다. order-service가 product-service를 직접 HTTP로 호출하기 때문입니다. 하나가 느려지면 다른 하나도 기다립니다. 하나가 멈추면 다른 하나도 멈춥니다.

생각해 보면 이것은 MSA의 목표, 즉 “각 서비스를 독립적으로 배포하고 장애를 격리한다”에 정면으로 위배됩니다.

4.1.1 카운터 대기 vs 진동벨

카페에서 커피를 주문하는 두 가지 방식으로 비교하겠습니다.

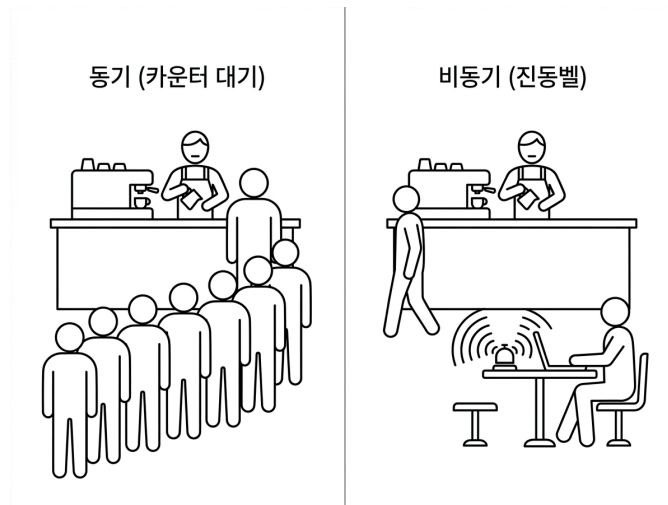


그림 0-1: 동기 vs 비동기 통신

카운터 대기 방식 (동기적)

카운터 앞에 서서 “아메리카노 하나요”라고 주문합니다. 바리스타가 커피를 만드는 동안 카운터 앞에서 기다립니다. 내 커피가 나와야 다음 손님이 주문할 수 있습니다. 바리스타가 원두를 갈다가 머신이 고장 나면? 뒤에 줄 선 손님 전부가 멈춥니다.

진동벨 방식 (비동기적)

“아메리카노 하나요”라고 주문하면 진동벨을 받고 자리에 앉습니다. 바리스타는 주문을 순서대로 처리하고, 커피가 완성되면 벨이 울립니다. 나는 기다리는 동안 다른 일을 할 수 있고, 뒤에 줄 선 손님도 바로 주문할 수 있습니다. 머신이 잠깐 멈춰도 주문 목록은 사라지지 않습니다.

MSA에서 동기 통신의 문제는 더 심각합니다. 하나가 느려지면 전체가 느려지는 연쇄 장애가 발생합니다.

4.2 Kafka : 메시지를 전달하는 우체국

Apache Kafka: 서비스 사이에 메시지를 안전하게 저장하고 전달하는 분산 메시지 플랫폼입니다. 발행자와 구독자가 동시에 실행 중이지 않아도 메시지가 유실되지 않습니다.

Kafka를 코드로 보기 전, 꼭 알아야 할 세 가지 개념이 있습니다.

4.2.1 토픽, 프로듀서, 컨슈머

Kafka의 구조는 우체통과 비슷합니다. 메시지를 보내는 사람이 프로듀서, 메시지를 받는 사람이 컨슈머, 메시지가 쌓이는 공간이 토픽입니다.

토픽(Topic): 메시지가 저장되는 이름 붙은 채널입니다. 우체통의 투입구처럼 특정 주제의 메시지를 모아두는 공간입니다.

프로듀서(Producer): 토픽에 메시지를 보내는(발행하는) 쪽입니다. 우체통에 편지를 넣는 사람에 해당합니다.

컨슈머(Consumer): 토픽에서 메시지를 읽는(구독하는) 쪽입니다. 우체통에서 편지를 꺼내 읽는 사람에 해당합니다.

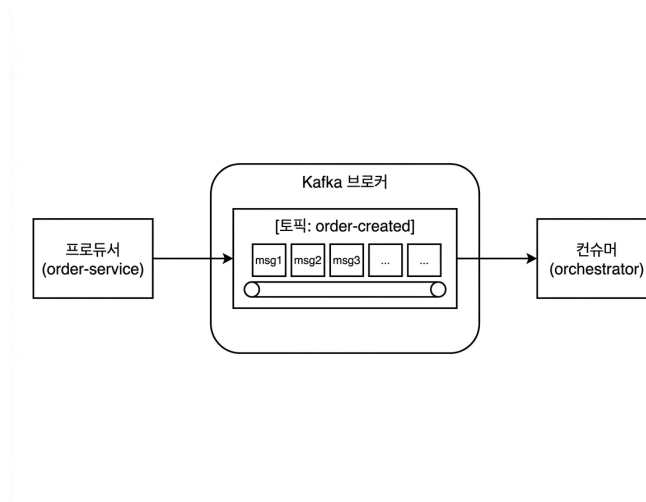


그림 0-2: 메시지 큐 개념

Kafka의 중요한 특성은, 컨슈머가 메시지를 읽어도 토픽에서 삭제되지 않는다는 점입니다. 보존 기간(기본 7일)이 지나야 삭제됩니다. 컨슈머는 오프셋(자신이 어디까지 읽었는지 기록한 위치)을 기억하기 때문에 재시작 후에도 읽지 않은 메시지부터 이어서 처리할 수 있습니다.

RabbitMQ 같은 전통적인 메시지 큐는 컨슈머가 메시지를 읽으면 큐에서 바로 삭제됩니다. 반면 **Kafka** 는 읽어도 메시지가 남아 있습니다. 덕분에 여러 컨슈머가 같은 메시지를 각자의 속도로 읽을 수 있고, 장애 후 재처리도 가능합니다.

4.2.2 컨슈머 그룹

product-service를 2대로 늘려서 운영한다고 가정해 봅시다. “재고 1개 차감” 메시지가 들어왔는데 두 인스턴스가 모두 처리하면 재고가 2개 줄어듭니다. 이런 중복 처리를 방지하기 위해 컨슈머 그룹으로 묶습니다. 같은 그룹 안에서는 하나의 메시지가 한 인스턴스에만 전달됩니다.

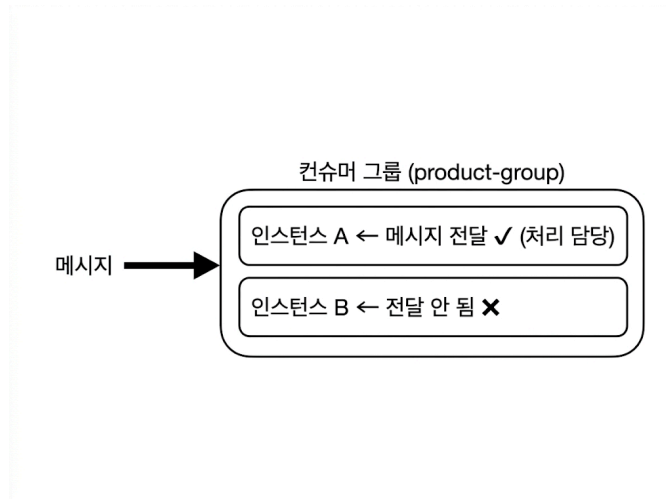


그림 0-3: 컨슈머 그룹

4.2.3 이번 챕터에서 사용하는 토픽 맵

이번 챕터에서 사용하는 Kafka 토픽 전체를 먼저 봅시다. 처음에는 낯설어 보이지만, 구현 과정에서 하나씩 다루게 됩니다.

총 9개의 토픽을 사용합니다. `command` 가 붙은 토픽은 orchestrator가 각 서비스에 내리는 명령입니다. `command` 가 없는 토픽은 각 서비스가 처리 결과를 orchestrator에 보고하는 이벤트입니다.

토픽	발행	구독	목적
order-created	order-service	orchestrator	새 주문 발생
decrease-product-command	orchestrator	product-service	재고 감소 명령
product-decreased	product-service	orchestrator	재고 감소 결과
create-delivery-command	orchestrator	delivery-service	배달 생성 명령
delivery-created	delivery-service	orchestrator	배달 생성 결과
complete-order-command	orchestrator	order-service	주문 완료 명령
cancel-order-command	orchestrator	order-service	주문 취소 명령 (롤백)
increase-product-command	orchestrator	product-service	재고 복구 명령 (롤백)
cancel-delivery-command	orchestrator	delivery-service	배달 취소 명령 (롤백)

4.3 Orchestration Saga : 지휘자가 흐름을 조율하다

챕터 1에서 소개한 Orchestration Saga를 이제 Kafka로 구현합니다. 지휘자(**orchestrator**)가 전체 흐름을 중앙에서 관리합니다. 각 서비스는 명령을 받아 처리하고 결과를 보고할 뿐, 다음 단계가 무엇인지 알 필요가 없습니다. 지휘자만 전체 악보를 알고 있습니다.

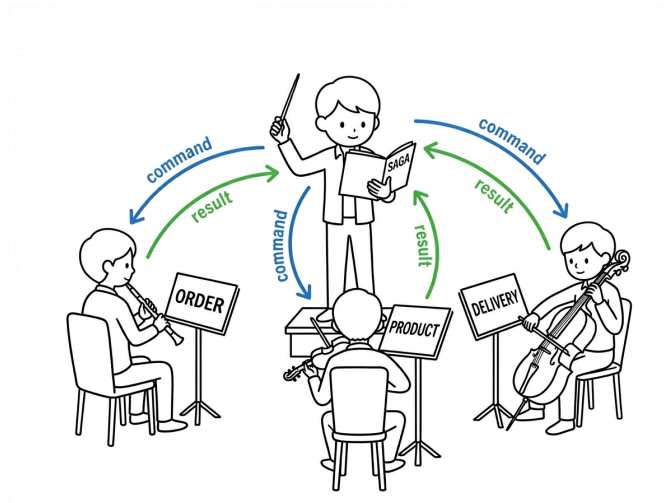


그림 0-5: Orchestration Saga 구조

4.3.1 주문 요청 성공 흐름

주문이 정상적으로 처리되는 흐름입니다. 클라이언트가 주문을 요청하면, orchestrator가 재고 감소 → 배달 생성 → 주문 완료 순서로 각 서비스에 명령을 보냅니다. 모든 단계가 성공하면 주문 상태가 COMPLETED로 바뀝니다.

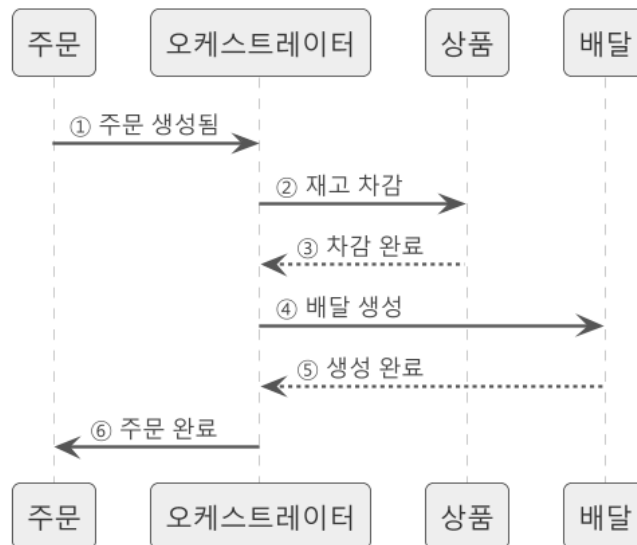


그림 0-6: 주문 성공 흐름 (Orchestration Saga)

단계	발행	토픽	수신	처리
1	클라이언트	/api/orders	order	주문 생성 요청 (PENDING)
2	order	order-created	orchestrator	주문 생성 확인
3	orchestrator	decrease-product-command	product	재고 감소 명령
4	product	product-decreased	orchestrator	감소 완료 확인
5	orchestrator	create-delivery-command	delivery	배달 생성 명령
6	delivery	delivery-created	orchestrator	생성 완료 확인
7	orchestrator	complete-order-command	order	주문 생성 완료 (COMPLETED)

4.3.2 주문 요청 실패 흐름 (롤백)

중간에 실패가 발생하면 orchestrator가 이미 처리된 단계를 역순으로 되돌립니다. 핵심은 “이미 처리된 것만 롤백” 한다는 점입니다.

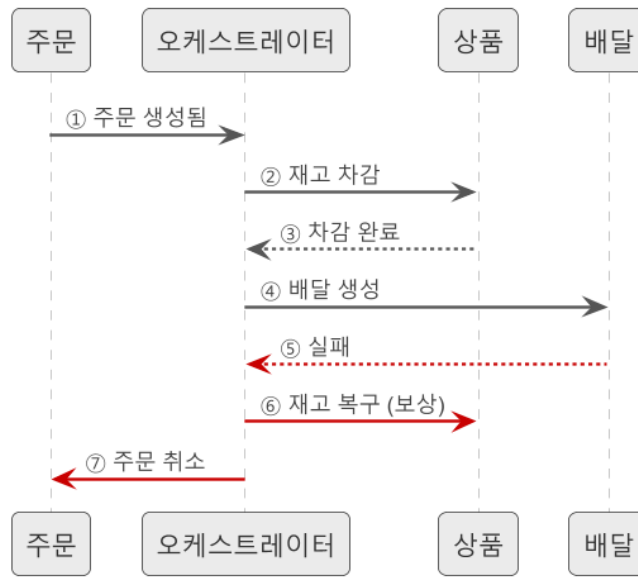


그림 0-8: 주문 실패 시 롤백 흐름

재고 감소 실패 시

재고가 부족하면 product-service가 실패를 보고합니다. 아직 배달은 생성되지 않았으므로, 주문만 취소하면 됩니다.

발행	토픽	수신	처리
product	product-decreased {success: false}	orchestrator	실패 감지
orchestrator	increase-product-command	product	이미 감소된 상품만 재고 복구
orchestrator	cancel-order-command	order	주문 취소 명령

배달 생성 실패 시

배달 실패 시 재고 복구와 주문 취소를 함께 합니다.

발행	토픽	수신	처리
delivery	delivery-created {success: false}	orchestrator	실패 감지
orchestrator	increase-product-command	product	이미 감소된 상품만 재고 복구
orchestrator	cancel-order-command	order	주문 취소 명령

4.4 order-service : 동기 호출 제거, 이벤트 발행

챕터 2:3에서 order-service는 product-service와 delivery-service를 직접 REST로 호출했습니다. 이번 챕터에서는 이 호출을 모두 제거하고 Kafka 이벤트를 발행으로 교체합니다.

4.4.1 패키지 구조

```

order-service/src/main/
├── resources/
│   └── application-prod.properties ← [참고] Kafka 설정 추가
├── java/.../
│   ├── adapter/
│   │   ├── message/
│   │   │   ├── OrderCreatedEvent.java ← [참고] 주문 생성 이벤트 DTO
│   │   │   ├── OrderCancelledEvent.java ← [참고] 주문 취소 이벤트 DTO
│   │   │   ├── CompleteOrderCommand.java ← [참고] 완료 명령 수신 DTO
│   │   │   └── CancelOrderCommand.java ← [참고] 취소 명령 수신 DTO
│   │   └── producer/
│   │       └── OrderEventProducer.java ← [작성] order-created, order-cancelled 발행
│   │   └── consumer/
│   │       └── OrderCommandConsumer.java ← [작성] complete/cancel 명령 수신
│   └── usecase/
│       └── OrderService.java ← [작성] createOrder 수정 (Kafka 발행으로 교체)

```

4.4.2 의존성 추가

`spring-boot-starter-kafka` 와 JSON 직렬화용 `jackson-databind` 를 추가합니다.

[참고] 챕터 3 `build.gradle` 에 다음 두 줄을 추가합니다.

```
implementation 'org.springframework.boot:spring-boot-starter-kafka'
implementation 'com.fasterxml.jackson.core:jackson-databind'
```

4.4.3 Kafka 설정

`application-dev.properties` 와 `application-prod.properties` 모두에 Kafka 설정을 추가합니다. Kafka 주소는 환경변수로 주입받습니다. > `group-id` 는 컨슈머 그룹 이름으로, 같은 그룹끼리는 메시지를 나눠 받고, 다른 그룹이면 같은 메시지를 각각 받습니다.

[참고] `application-prod.properties`

```
# ===== Kafka =====
spring.kafka.bootstrap-servers=${SPRING_KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
spring.kafka.consumer.group-id=order-service      # 이 서비스의 컨슈머 그룹 이름
# 생략 ...
```

dev와 prod의 Kafka 설정은 동일합니다. `bootstrap-servers` 의 기본값이 `localhost:9092` 이므로 로컬에서는 그대로, K8s에서는 환경변수로 Kafka 주소를 주입합니다.

4.4.4 KafkaConfig : JSON 메시지 변환

`JacksonJsonMessageConverter` 빈을 등록합니다. 이 설정은 모든 Kafka 사용 서비스(order, product, delivery, orchestrator)에 동일하게 추가합니다.

[참고] `core/config/KafkaConfig.java`

```
@Configuration
public class KafkaConfig {

    @Bean
    public RecordMessageConverter recordMessageConverter() {
        return new JacksonJsonMessageConverter();
    }
}
```

4.4.5 이벤트 DTO

Kafka 메시지로 전송할 데이터를 DTO로 정의합니다. 주문 생성 이벤트에는 주문 ID, 사용자 ID, 배달 주소, 주문 상품 목록이 필요합니다.

[참고] `adapter/message/OrderCreatedEvent.java`

```

@Getter
@NoArgsConstructor
@AllArgsConstructor
public class OrderCreatedEvent {
    private int orderId;
    private int userId;
    private String address;
    private List<OrderItem> orderItems;

    @Getter
    @NoArgsConstructor
    @AllArgsConstructor
    public static class OrderItem {
        private int productId;
        private int quantity;
        private Long price;
    }
}

```

4.4.6 KafkaTemplate : 메시지 발행

이벤트를 Kafka 토픽으로 발행하는 프로듀서입니다. `KafkaTemplate` 이 직렬화와 전송을 처리합니다.

다음 코드를 추가하세요.

[작성] `adapter/producer/OrderEventProducer.java`

```

@Component
@RequiredArgsConstructor
public class OrderEventProducer {
    private final KafkaTemplate<String, Object> kafkaTemplate;

    // 주문 생성 이벤트 발행
    public void publishOrderCreated(OrderCreatedEvent event) {
        kafkaTemplate.send("order-created", event);
    }

    // 주문 취소 이벤트 발행
    public void publishOrderCancelled(OrderCancelledEvent event) {
        kafkaTemplate.send("order-cancelled", event);
    }
}

```

```

    }
}

```

4.4.7 OrderService 변경 : 핵심 차이

이전 챕터와 비교하여 가장 크게 달라지는 부분은 동기 호출 대신 Kafka 이벤트를 발행하고 즉시 PENDING 상태로 반환한다는 점입니다. 아래 createOrder 메서드를 구현합니다.

[작성] `usecase/OrderService.java`

```

@Override
@Transactional
public OrderResponse createOrder(int userId, List<OrderRequest.OrderItemDTO> orderItems, String
address) {
    // 1. 주문 생성 (PENDING 상태)
    Order createdOrder = orderRepository.save(Order.create(userId));
    final int orderId = createdOrder.getId();

    // 2. 주문 아이템 저장
    List<OrderItem> createdOrderItems = orderItems.stream()
        .map(item -> OrderItem.create(orderId, item.productId(), item.quantity(), item.price()))
        .toList();
    orderItemRepository.saveAll(createdOrderItems);

    // 3. Kafka로 주문 생성 이벤트 발행 - 수정: REST 직접 호출 -> Kafka 이벤트 발행
    List<OrderCreatedEvent.OrderItem> messageItems = orderItems.stream()
        .map(item -> new OrderCreatedEvent.OrderItem(item.productId(), item.quantity(), item.price()))
        .toList();
    orderEventProducer.publishOrderCreated(new OrderCreatedEvent(orderId, userId, address,
messageItems));

    return OrderResponse.from(createdOrder, createdOrderItems);
}

```

4.4.8 @KafkaListener : complete/cancel 명령 처리

orchestrator가 최종 결정을 내리면 Kafka로 완료(`complete-order-command`) 또는 취소(`cancel-order-command`) 명령을 발행합니다. order-service는 이 명령을 수신하여 주문 상태를 COMPLETED 또는 CANCELLED로 변경합니다.

[작성] `adapter/consumer/OrderCommandConsumer.java` - 다음 코드를 추가하세요.

```

@Component
@RequiredArgsConstructor
public class OrderCommandConsumer {

    private final OrderService orderService;

    // 주문 완료 명령 수신
    @KafkaListener(topics = "complete-order-command", groupId = "order-service")
    public void completeOrderCommand(CompleteOrderCommand command) {
        orderService.completeOrder(command.getOrderId());
    }

    // 주문 취소 명령 수신
    @KafkaListener(topics = "cancel-order-command", groupId = "order-service")
    public void cancelOrderCommand(CancelOrderCommand command) {
        orderService.cancelOrder(command.getOrderId());
    }
}

```

4.5 product-service · delivery-service : Kafka Consumer/Producer 추가

product-service는 재고 감소/복구 명령을 Kafka로 받고, 결과를 Kafka로 발행합니다. REST API는 그대로 유지됩니다. Kafka는 기존 API 위에 새로운 통신 채널을 추가하는 것입니다.

4.5.1 패키지 구조

```

adapter/
├── consumer/
│   └── ProductCommandConsumer.java # [참고] @KafkaListener (decrease/increase 명령 수신)
├── producer/
│   └── ProductEventProducer.java # [참고] KafkaTemplate 발행
└── message/
    ├── DecreaseProductCommand.java # [참고] 수신 DTO (재고 감소 명령)
    ├── IncreaseProductCommand.java # [참고] 수신 DTO (재고 복구 명령)
    └── ProductDecreasedEvent.java # [참고] 발행 DTO (재고 감소 결과)

```

4.5.2 메시지 DTO 정의

Kafka로 전송되는 메시지는 Java 객체를 JSON으로 직렬화한 것입니다. 각 서비스는 자신이 발행하거나 수신하는 메시지에 해당하는 DTO 클래스를 가지고 있습니다.

토픽	DTO 클래스	주요 필드
increase-product-command	IncreaseProductCommand	orderId, productId, quantity, price
product-decreased	ProductDecreasedEvent	orderId, productId, quantity, success
create-delivery-command	CreateDeliveryCommand	orderId, address
cancel-delivery-command	CancelDeliveryCommand	orderId
delivery-created	DeliveryCreatedEvent	orderId, deliveryId, success
complete-order-command	CompleteOrderCommand	orderId
cancel-order-command	CancelOrderCommand	orderId
order-cancelled	OrderCancelledEvent	orderId, List<OrderItem> (productId, quantity, price)

product-service와 delivery-service의 Consumer/Producer는 order-service와 동일한 패턴을 따릅니다.

전체 코드는 GitHub에서 확인하세요.

4.6 orchestrator : 워크플로우 조율 서비스 구현 (핵심)

orchestrator는 이번 챕터에서 새로 추가되는 서비스입니다. REST API는 없고, Kafka 이벤트만 처리합니다. order, product, delivery 서비스의 Kafka 흐름을 중앙에서 조율하는 지휘자입니다.

4.6.1 패키지 구조

```
orchestrator/src/main/java/.../
├── message/
│   ├── OrderCreatedEvent.java      ← [참고] order-service에서 수신
│   ├── DecreaseProductCommand.java ← [참고] product-service에 발행
│   └── IncreaseProductCommand.java ← [참고] product-service에 발행 (롤백)
```

```

| └─ ProductDecreasedEvent.java    ← [참고] product-service에서 수신
| └─ CreateDeliveryCommand.java    ← [참고] delivery-service에 발행
| └─ DeliveryCreatedEvent.java     ← [참고] delivery-service에서 수신
| └─ CompleteOrderCommand.java     ← [참고] order-service에 발행
| └─ OrderCancelledEvent.java      ← [참고] order-service에서 수신
| └─ CancelOrderCommand.java       ← [참고] order-service에 발행 (롤백)
| └─ CancelDeliveryCommand.java    ← [참고] delivery-service에 발행 (롤백)
└─ handler/
    └─ OrderOrchestrator.java      ← [작성] 전체 워크플로우 조율

```

4.6.2 의존성

Kafka 메시지를 주고받기 위한 `spring-boot-starter-kafka` 와 JSON 직렬화를 위한 `jackson-databind` 를 추가합니다.

[참고] `build.gradle`

```

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-kafka'
    implementation 'com.fasterxml.jackson.core:jackson-databind'
}

```

4.6.3 OrderOrchestrator

OrderOrchestrator는 세 단계로 동작합니다.

1. **orderCreated** – 주문 생성 이벤트를 받아 재고 감소 명령을 발행합니다.
2. **productDecreased** – 재고 감소 결과를 받아, 성공이면 배달 생성 명령을, 실패면 주문 취소 명령을 발행합니다.
3. **deliveryCreated** – 배달 생성 결과를 받아, 성공이면 주문 완료 명령을, 실패면 재고 복구 + 주문 취소 명령을 발행합니다.

각 단계를 하나씩 살펴보겠습니다.

[작성] `handler/OrderOrchestrator.java` – OrderOrchestrator 클래스

```

@Component
@RequiredArgsConstructor
public class OrderOrchestrator {
    private final KafkaTemplate<String, Object> kafkaTemplate;

```

```
private final Map<Integer, WorkflowState> states = new ConcurrentHashMap<>(); // Kafka 리스너가 멀티
스레드로 동작하므로 동시성 보장 필요

// 아래 1~3단계 메서드와 WorkflowState 내부 클래스를 추가
}
```

1단계: 주문 생성 이벤트 수신 (orderCreated)



그림 0-12: 1단계: 주문 생성 이벤트 수신

[작성] `handler/OrderOrchestrator.java`

```
@KafkaListener(topics = "order-created", groupId = "orchestrator")
public void orderCreated(OrderCreatedEvent event) {
    int orderId = event.getOrderId();
    List<OrderCreatedEvent.OrderItem> items = List.copyOf(event.getOrderItems());

    // 주문별 워크플로우 상태 생성
    states.put(orderId, new WorkflowState(orderId, event.getAddress(), items));

    // 각 상품에 재고 감소 명령 발행
    for (OrderCreatedEvent.OrderItem item : items) {
        kafkaTemplate.send(
            "decrease-product-command",
            String.valueOf(orderId),
            new DecreaseProductCommand(
                orderId,
                item.getProductId(),
                item.getQuantity(),
                item.getPrice()
            )
        );
    }
}
```

orchestrator가 주문 생성(`order-created`) 이벤트를 받으면, 주문 ID별로 `WorkflowState`를 생성합니다. 그리고 각 상품에 대해 재고 감소 명령(`decrease-product-command`)을 발행합니다.

2단계: 재고 차감 결과 수신 (productDecreased)

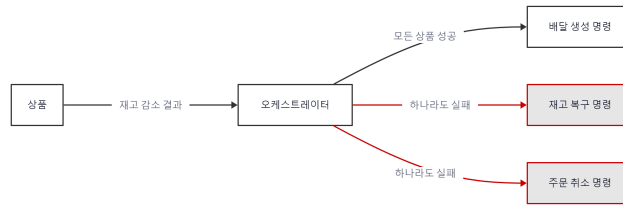


그림 0-13: 2단계: 재고 차감 결과 수신

[작성] `handler/OrderOrchestrator.java`

```

@KafkaListener(topics = "product-decreased", groupId = "orchestrator")
public void productDecreased(ProductDecreasedEvent event) {
    int orderId = event.getOrderId();
    WorkflowState state = states.get(orderId);
    if (state == null) return;

    // 실패: 이미 감소된 상품만 복구 후 주문 취소
    if (!event.isSuccess()) {
        for (OrderCreatedEvent.OrderItem item : state.items) {
            if (state.decreasedProductIds.contains(item.getProductId())) {
                kafkaTemplate.send(
                    "increase-product-command",
                    String.valueOf(orderId),
                    new IncreaseProductCommand(
                        orderId,
                        item.getProductId(),
                        item.getQuantity(),
                        item.getPrice()
                    )
                );
            }
        }
        kafkaTemplate.send(
            "cancel-order-command",
            String.valueOf(orderId),
            new CancelOrderCommand(orderId)
        );
        states.remove(orderId);
        return;
    }
}

```

```

// 성공: 감소 완료된 상품 ID 기록
state.decreasedProductIds.add(event.getProductId());
state.processed++;

// 모든 상품 감소 완료 → 배달 생성 명령 발행
if (state.processed == state.getItems().size()) {
    kafkaTemplate.send(
        "create-delivery-command",
        String.valueOf(orderId),
        new CreateDeliveryCommand(orderId, state.address)
    );
}
}

```

각 상품의 재고 차감 결과가 하나씩 돌아옵니다. 실패 시에는 이미 차감된 상품(`decreasedProductIds`)에 기록된 것)만 골라 재고를 복구합니다. 그런 다음 주문 취소 명령(`cancel-order-command`)으로 주문을 취소합니다. 모든 상품이 성공하면 배달 생성 명령(`create-delivery-command`)을 발행합니다.

`decreasedProductIds` (Set)는 어떤 상품이 성공했는지, `processed` (int)는 몇 개가 완료됐는지를 추적합니다. 실패 시에는 성공한 상품만 골라 복구해야 하고, 전체 완료 판단에는 개수 비교가 필요하기 때문에 둘 다 필요합니다.

3단계: 배달 생성 결과 수신 (deliveryCreated)

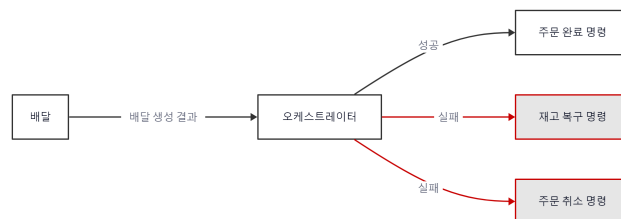


그림 0-14: 3단계: 배달 생성 결과 수신

[작성] `handler/OrderOrchestrator.java`

```

@KafkaListener(topics = "delivery-created", groupId = "orchestrator")
public void deliveryCreated(DeliveryCreatedEvent event) {
    int orderId = event.getOrderId();
    WorkflowState state = states.get(orderId);
}

```

```

    if (state == null) return;

    // 실패: 전체 재고 복구 후 주문 취소
    if (!event.isSuccess()) {
        for (OrderCreatedEvent.OrderItem item : state.items) {
            if (state.decreasedProductIds.contains(item.getProductId())) {
                kafkaTemplate.send(
                    "increase-product-command",
                    String.valueOf(orderId),
                    new IncreaseProductCommand(
                        orderId,
                        item.getProductId(),
                        item.getQuantity(),
                        item.getPrice()
                    )
                );
            }
        }
        kafkaTemplate.send(
            "cancel-order-command",
            String.valueOf(orderId),
            new CancelOrderCommand(orderId)
        );
        states.remove(orderId);
        return;
    }

    // 성공: 주문 완료 명령 발행
    kafkaTemplate.send(
        "complete-order-command",
        String.valueOf(orderId),
        new CompleteOrderCommand(orderId)
    );
    states.remove(orderId); // 워크플로우 종료
}

```

배달 생성이 실패하면 차감된 재고를 모두 복구하고 주문을 취소합니다. 성공하면 주문 완료 명령을 발행합니다.

4.6.4 WorkflowState : 주문별 진행 상태 추적

orchestrator는 진행 중인 주문의 상태를 메모리에 보관합니다. 어떤 상품의 재고가 이미 감소했는지 추적하여, 실패 시 처리된 항목만 롤백합니다.

WorkflowState: orchestrator가 주문 한 건의 진행 상태를 추적하기 위해 사용하는 내부 객체입니다. 주문 ID, 상품 목록, 이미 처리된 상품 ID 등을 보관합니다.

[작성] `handler/OrderOrchestrator.java` 내부 클래스

```
@Data
private static class WorkflowState {
    private final int orderId;
    private final String address;
    private final List<OrderCreatedEvent.OrderItem> items;
    private int processed = 0; // 처리 완료된 상품 수
    private final Set<Integer> decreasedProductIds = ConcurrentHashMap.newKeySet(); // 재고 감소 완료된
    상품 ID
}
```

모든 서비스의 Kafka 연동 코드가 완성되었습니다. 이제 Kubernetes에 Kafka와 orchestrator를 추가하고 전체 시스템을 배포합니다.

4.7 Kubernetes : Kafka와 orchestrator 배포

챕터 3과 비교하여 이번 챕터에서 K8s에 새로 추가되는 것은 **Kafka**와 **orchestrator** 두 가지입니다. 기존 서비스의 ConfigMap에도 Kafka 주소를 추가해야 합니다.

4.7.1 Kafka

기존 Kafka는 ZooKeeper라는 별도 서비스가 필요했지만, **KRaft(Kafka Raft)** 모드를 사용하면 ZooKeeper 없이 Kafka 자체적으로 메타데이터를 관리합니다. `confluentinc/cp-kafka:7.5.0` 이미지를 사용합니다.

K8s 리소스	파일	역할	비고
Deployment	<code>kafka-deploy.yml</code>	KRaft 모드 Kafka 브로커 실행	<code>KAFKA_PROCESS_ROLES:</code> <code>broker,controller</code>
Service	<code>kafka-service.yml</code>	브로커(9092)·컨트롤러(9093) 포트 노출	Spring Boot는 <code>kafka-service:9092</code> 로 접근

4.7.2 orchestrator

orchestrator는 이번 챕터에서 새로 추가되는 서비스입니다. REST API 없이 Kafka 이벤트만 처리하므로 Service는 필요 없습니다.

K8s 리소스	파일	역할
ConfigMap	<code>orchestrator-configmap.yml</code>	KAFKA_BOOTSTRAP_SERVERS 설정
Deployment	<code>orchestrator-deploy.yml</code>	orchestrator Pod 실행

4.7.3 기존 서비스 ConfigMap 수정

기존 서비스(order, product, delivery)의 ConfigMap에 `KAFKA_BOOTSTRAP_SERVERS: kafka-service:9092`를 추가합니다.

전체 YAML은 GitHub에서 확인하세요.

4.8 실행 및 결과 확인

4.8.1 이미지 빌드

Minikube 내부에 이미지를 빌드합니다. 챕터 3 대비 orchestrator 서비스가 새로 추가됩니다.

```
minikube image build -t metacoding/db:2 ./db
minikube image build -t metacoding/gateway:2 ./gateway
minikube image build -t metacoding/order:2 ./order
minikube image build -t metacoding/product:2 ./product
minikube image build -t metacoding/user:2 ./user
minikube image build -t metacoding/delivery:2 ./delivery
minikube image build -t metacoding/orchestrator:2 ./orchestrator
```

4.8.2 배포 순서

Kafka가 준비되기 전에 서비스가 시작되면 연결 오류가 발생합니다. Kafka를 먼저 배포하고 ready 상태를 확인한 다음 나머지를 배포합니다.

1. 네임스페이스 생성 (최초 1회)

kubectl create namespace metacoding

2. Kafka 먼저 배포

kubectl apply -f k8s/kafka

3. Kafka가 준비될 때까지 대기

kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

4. 나머지 서비스 배포

kubectl apply -f k8s/db

kubectl apply -f k8s/order

kubectl apply -f k8s/product

kubectl apply -f k8s/user

kubectl apply -f k8s/delivery

kubectl apply -f k8s/gateway

kubectl apply -f k8s/orchestrator

5. Ingress 활성화 (최초 1회)

minikube addons enable ingress

```

$ kubectl create namespace metacoding
namespace/metacoding created

$ kubectl apply -f k8s/kafka
deployment.apps/kafka-deploy created
service/kafka-service created

$ kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s
pod/kafka-deploy-69fc99fc86-85vhw condition met

$ kubectl apply -f k8s/db
configmap/db-configmap created
deployment.apps/db-deploy created
secret/db-secret created
service/db-service created

$ kubectl apply -f k8s/order
configmap/order-configmap created
deployment.apps/order-deploy created
secret/order-secret created
service/order-service created

$ kubectl apply -f k8s/product
configmap/product-configmap created
deployment.apps/product-deploy created
secret/product-secret created
service/product-service created

$ kubectl apply -f k8s/user
configmap/user-configmap created
deployment.apps/user-deploy created
secret/user-secret created
service/user-service created

$ kubectl apply -f k8s/delivery
configmap/delivery-configmap created
deployment.apps/delivery-deploy created
secret/delivery-secret created
service/delivery-service created

$ kubectl apply -f k8s/gateway
deployment.apps/gateway-deploy created
service/gateway-service created
ingress.networking.k8s.io/gateway-ingress created

$ kubectl apply -f k8s/orchestrator
configmap/orchestrator-configmap created
deployment.apps/orchestrator-deploy created

```

그림 0-17: Kafka 및 서비스 배포 실행

모든 Pod가 Running 상태인지 확인합니다.

kubectl get pods -n metacoding

```

$ kubectl get pods -n metacoding
NAME                                READY   STATUS    RESTARTS   AGE
db-deploy-5c94cbfb5f-sbjdj          1/1     Running   0           3m48s
delivery-deploy-56cfc89bdd-5znd7    1/1     Running   1 (102s ago) 3m46s
gateway-deploy-5d4c9c94c-v568l      1/1     Running   0           3m45s
kafka-deploy-69fc99fc86-85vhw       1/1     Running   0           4m37s
orchestrator-deploy-65bdd754c6-wstwj 1/1     Running   0           3m44s
order-deploy-665d89c8fc-tnntz       1/1     Running   1 (110s ago) 3m48s
product-deploy-6f5745c6b9-vvw6r     1/1     Running   1 (103s ago) 3m47s
user-deploy-5d786978bd-vlm8w        1/1     Running   1 (112s ago) 3m47s

```

그림 0-18: Pod 상태 확인 (kubectl get pods)

4.8.3 서비스 접근

Ingress를 통해 외부에서 접속하려면 **minikube tunnel** 을 실행합니다.

```
minikube tunnel
```

터널이 실행되면 <http://127.0.0.1:80> 로 gateway-service에 접속할 수 있습니다.

4.8.4 비동기 흐름 테스트

AirPods (productId=3)를 주문합니다.

POST <http://127.0.0.1:80/api/orders>

```
{
  "address": "Addr 4",
  "orderItems": [
    {
      "productId": 3,
      "quantity": 2,
      "price": 300000
    }
  ]
}
```

JSON Raw 헤더 테스트 결과

응답 본문

```
1 - {
2   "status": 200,
3   "msg": "성공",
4   "body": {
5     "id": 4,
6     "userId": 1,
7     "status": "PENDING",
8     "createdAt": "2026-03-20T10:51:56.815246726",
9     "updatedAt": "2026-03-20T10:51:56.815273872",
10    "orderItems": [
11      {
12        "createdAt": "2026-03-20T10:51:57.198623723",
13        "id": 4,
14        "orderId": 4,
15        "price": 300000,
16        "productId": 3,
17        "quantity": 2,
18        "updatedAt": "2026-03-20T10:51:57.19865665"
19      }
20    ]
21  }
```

그림 0-19: 주문 생성 응답 (PENDING 상태)

챕터 3과 다르게 즉시 **PENDING** 상태로 반환됩니다. Kafka 이벤트가 처리되면 상태가 **COMPLETED**로 변경됩니다. 잠시 후 주문 상태를 다시 조회합니다.

GET <http://127.0.0.1:80/api/orders/4>

JSON	Raw	헤더	테스트 결과
응답 본문			
1	*		{
2			"status": 200,
3			"msg": "성공",
4	*		"body": {
5			"id": 4,
6			"userId": 1,
7			"status": "COMPLETED",
8			"createdAt": "2026-03-20T10:51:57",
9			"updatedAt": "2026-03-20T10:52:01",
10	*		"orderItems": [
11	*		{
12			"createdAt": "2026-03-20T10:51:57",
13			"id": 4,
14			"orderId": 4,
15			"price": 300000,
16			"productId": 3,
17			"quantity": 2,
18			"updatedAt": "2026-03-20T10:51:57"
19			}
20			]
21			}

그림 0-20: 주문 완료 확인 (COMPLETED 상태)

4.8.5 롤백 확인 : 품질 상품 주문

iPhone 15(productId=2, 재고 0)를 주문하면 product-service에서 재고 감소가 실패하고, orchestrator가 자동으로 롤백을 시작합니다.

POST http://127.0.0.1:80/api/orders

```
{
  "address": "Addr 5",
  "orderItems": [
    {
      "productId": 2,
      "quantity": 1,
      "price": 1300000
    }
  ]
}
```

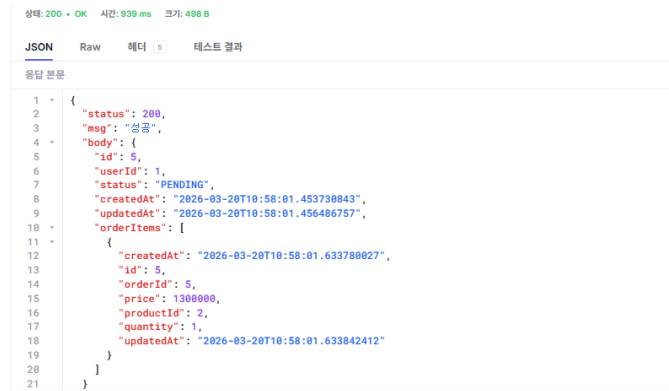


그림 0-21: 품질 상품 주문 요청

잠시 후 상태를 확인하면 **CANCELLED** 가 됩니다.

GET <http://127.0.0.1:80/api/orders/5>

orchestrator 로그에서 롤백 과정을 확인할 수 있습니다.



그림 0-22: orchestrator 롤백 로그 확인

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

kubectl delete all --all -n metacoding

정리

이번 챕터에서 만든 것을 정리합니다.

- 동기 REST 호출을 **Kafka** 비동기 이벤트로 교체하여 서비스 간 결합을 끊었습니다.
- **orchestrator** 서비스가 Kafka 토픽을 통해 전체 워크플로우를 조율합니다.

- `ConcurrentHashMap` 으로 주문별 진행 상태를 추적하고, 실패 시 이미 처리된 단계만 자동 롤백합니다.
- `order-service`는 주문 생성 즉시 `PENDING` 상태로 반환하고, 처리 완료 후 `COMPLETED` 로 갱신됩니다.

이제 `product-service`가 잠시 다운되어도 `order-service`는 이벤트를 Kafka에 올려두고 즉시 반환합니다. `product-service`가 복구되면 토픽에서 메시지를 읽어 처리합니다. 두 서비스가 강하게 결합되어 있던 문제가 해결됐습니다.

하지만 사용자 입장에서 불편한 점이 남아 있습니다. 주문이 `COMPLETED` 가 됐다는 것을 알기 위해 직접 주기적으로 조회(폴링)해야 합니다. 배달도 여전히 주문과 동시에 완료되어, 실제 배달 완료 시점을 반영하지 못합니다.

다음 챕터에서는 이 두 가지를 해결합니다. 배달 기사가 완료 API를 호출하여 실제 배달 완료를 처리하고, WebSocket으로 클라이언트에게 주문 완료를 실시간 Push로 알립니다.

챕터 5. 실시간 알림 - 주문 완료를 즉시 전달하다

chap04 · 실행 환경: 컨테이너 · WebSocket(STOMP/SockJS) · 배달 완료 라이프사이클 이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/chap04> 에서 확인할 수 있습니다.

학습 목표

- 폴링의 문제점을 이해하고 WebSocket의 장점을 설명한다.
- delivery-service에 배달 완료 API를 추가하고 배달 완료 이벤트를 발행한다.
- orchestrator가 배달 완료 이벤트를 받아 주문 완료 명령을 발행하도록 수정한다.
- order-service에 WebSocket을 설정하고 주문 완료 시 사용자에게 실시간 알림을 보낸다.
- 전체 시스템을 통합 테스트한다.

5.1 챕터 4가 남긴 두 가지 숙제

챕터 4를 마치면서 두 가지 한계를 이야기했습니다.

첫 번째: 사용자가 주문 완료를 알려면 직접 계속 조회해야 합니다. 동기 방식이라면 응답에 결과가 바로 담겨오지만, 비동기 방식에서는 결과가 나중에 정해집니다. 주문 직후 **PENDING** 상태를 받는 사용자는 “언제 완료되지?”라며 새로고침을 반복합니다. 음식 배달 앱에서 “지금 어디쯤이지?” 궁금해서 5초마다 앱을 열어보는 상황과 같습니다. 앱이 알아서 알려주면 좋겠지만, 그런 기능이 아직 없으니 직접 확인하는 수밖에 없습니다. 이것을 **폴링(Polling)** 이라고 합니다.

폴링(Polling): 클라이언트가 서버에 주기적으로 “변경된 거 있어?” 하고 반복 요청하여 상태를 확인하는 방식입니다. 서버가 먼저 알려주지 않으므로, 변화가 없어도 계속 요청이 발생합니다.

두 번째: delivery-service에서 배달이 생성되면 자동으로 완료 처리됩니다. 실제 서비스라면 배달 기사가 물건을 전달한 시점에 완료가 되어야 합니다.

이번 챕터에서는 이 두 숙제를 동시에 해결합니다. 배달 기사가 API를 호출하면 그 시점에 배달이 완료됩니다. 동시에 WebSocket으로 사용자에게 즉시 알림을 보냅니다. 이것이 완성된 주문 처리 시스템의 마지막 퍼즐입니다.

5.2 WebSocket : 폴링의 한계를 넘다

5.2.1 왜 폴링이 문제인가?

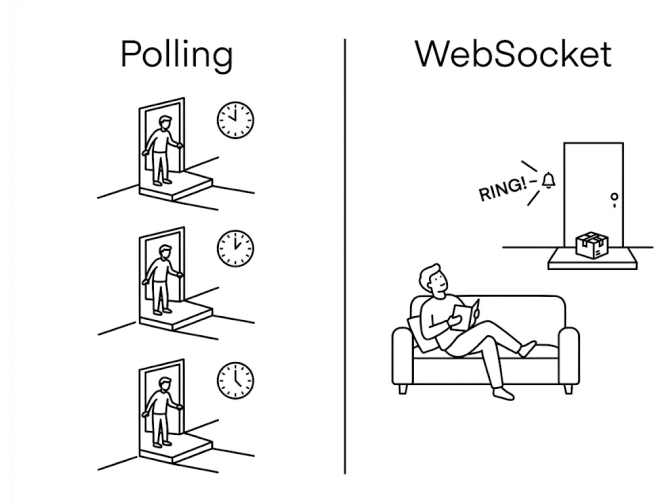


그림 0-1: 폴링 vs WebSocket

주문 완료 여부를 확인하는 방법은 크게 두 가지입니다.

폴링(Polling) 방식은 택배를 기다리며 현관문을 5분마다 직접 여는 것과 같습니다. 클라이언트가 서버에 반복적으로 “완료됐나요?”를 물어봅니다.

WebSocket 방식은 초인종을 다는 것과 같습니다. 택배가 도착하면 초인종이 울려 알려주듯, 서버가 먼저 클라이언트에게 Push합니다.

WebSocket 위에 STOMP 프로토콜을 사용합니다. STOMP를 얹으면 “이 채널을 구독한다”, “이 채널에 메시지를 보낸다” 같은 발행-구독 구조를 쓸 수 있습니다. 클라이언트가 `/topic/orders/{userId}` 채널을 구독하면 해당 사용자에게만 알림이 전달됩니다.

5.3 배달 완료 라이프사이클 설계

구현 전, 챕터 4와 챕터 5에서 배달 상태 전이가 어떻게 달라지는지 먼저 비교합니다.

챕터 4는 배달이 생성되는 즉시 완료 처리됩니다. 현실에서는 배달 기사가 물건을 전달해야 완료인데, 이 단계가 빠져 있습니다.

챕터 4 (이전)



그림 0-2: 챕터 4의 배달 흐름 - 생성 즉시 완료 처리

챕터 5에서는 실제 택배처럼 접수하면 “배달 중(PENDING)”이 되고, 배달 기사가 수령인에게 전달한 뒤 “배달 완료(COMPLETED)” 버튼을 눌러야 비로소 완료가 됩니다.

챕터 5 (변경)

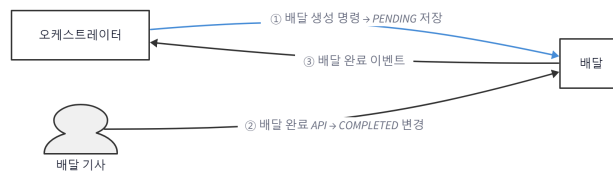


그림 0-3: 챕터 5의 배달 흐름 - 배달 기사 완료 API 호출 후 COMPLETED

이제 세 곳을 수정해야 합니다. `delivery-service`에 배달 완료 API를 추가하고, `orchestrator`에 배달 완료(`delivery-completed`) 토픽 처리를 추가하고, `order-service`에 WebSocket Push를 구현합니다.

5.4 delivery-service : 배달 완료 API 추가

5.4.1 패키지 구조

`delivery-service/src/main/java/.../`

```

├── domain/
│   └── Delivery.java          ← [참고] create() 초기 상태 PENDING으로 변경
├── usecase/
│   ├── CompleteDeliveryUseCase.java ← [참고] 배달 완료 인터페이스
│   └── DeliveryService.java      ← [작성] completeDelivery() 추가
├── web/
│   └── DeliveryController.java   ← [작성] PUT /{id}/complete 추가
  
```

```

└─ adapter/
  └─ message/
    └─ DeliveryCompletedEvent.java ← [참고] 배달 완료 이벤트 DTO
  └─ producer/
    └─ DeliveryEventProducer.java ← [작성] publishDeliveryCompleted() 추가

```

5.4.2 Delivery 엔티티 상태 전이

배달 생성 시 상태를 **PENDING** 으로 저장하고, 배달 기사가 완료 처리 시 **COMPLETED** 로 전이합니다. 이전에는 생성과 동시에 `complete()` 를 호출했지만, 이제는 명시적인 API 호출이 있어야 완료됩니다.

[참고] `domain/Delivery.java` - 동작 이해용입니다.

```

@Table(name = "delivery_tb")
public class Delivery {
    // 2장 Delivery.java 참조 - 필드 동일

    // 배달 주소 검증
    public static void validateAddress(String address) {
        if (address == null || address.isBlank()) {
            throw new Exception400("배달 주소는 필수입니다.");
        }
    }

    // 배달 생성 시 PENDING 상태 (배달 완료 API 대기)
    public static Delivery create(int orderId, String address) {
        return new Delivery(orderId, address, DeliveryStatus.PENDING);
    }

    public void complete() {
        this.status = DeliveryStatus.COMPLETED;
        this.updatedAt = LocalDateTime.now();
    }

    public void cancel() {
        validateCancelable();
        this.status = DeliveryStatus.CANCELLED;
        this.updatedAt = LocalDateTime.now();
    }
}

```

```

public void validateCancelable() {
    if (this.status == DeliveryStatus.CANCELLED) {
        throw new Exception400("배달이 이미 취소되었습니다.");
    }
}
}

```

아래 메서드를 DeliveryService에 추가합니다. 배달 상태를 COMPLETED로 변경하고 Kafka 이벤트를 발행합니다.

[작성] `usecase/DeliveryService.java`

```

@Override
@Transactional
public DeliveryResponse completeDelivery(int deliveryId) { // 추가
    Delivery findDelivery = deliveryRepository.findById(deliveryId)
        .orElseThrow(() -> new Exception404("배달 정보를 조회할 수 없습니다."));
    findDelivery.complete();
    deliveryEventProducer.publishDeliveryCompleted(new
    DeliveryCompletedEvent(findDelivery.getOrderId()));
    return DeliveryResponse.from(findDelivery);
}

```

5.4.3 배달 완료 컨트롤러

아래 엔드포인트를 DeliveryController에 추가합니다. 배달 기사가 호출하는 배달 완료 API입니다.

[작성] `web/DeliveryController.java` (완료 API 추가)

```

@PutMapping("/{deliveryId}/complete")
public ResponseEntity<?> completeDelivery(@PathVariable("deliveryId") int deliveryId) { // 추가
    return Resp.ok(completeDeliveryUseCase.completeDelivery(deliveryId));
}

```

5.4.4 delivery-completed 이벤트 발행

배달이 완료되면 Kafka 배달 완료(`delivery-completed`) 토픽에 이벤트를 발행합니다. orchestrator가 이 이벤트를 받아 다음 단계를 진행합니다.

아래 메서드를 기존 파일에 추가합니다.

[작성] `adapter/producer/DeliveryEventProducer.java` (publishDeliveryCompleted 추가)


```
public void publishDeliveryCompleted(DeliveryCompletedEvent event) { // 추가
    kafkaTemplate.send("delivery-completed", event);
}
```

5.5 orchestrator : delivery-completed 처리 추가

챕터 5에서는 배달 생성(`delivery-created`) 성공 시 바로 완료하지 않고 대기합니다. 이후 배달 완료(`delivery-completed`) 이벤트를 받았을 때 비로소 주문 완료 명령(`complete-order-command`)을 발행합니다.

5.5.1 deliveryCreated 수정 : 성공 시 대기

챕터 4에서는 배달 생성 성공 시 즉시 주문 완료 명령(`complete-order-command`)을 발행했지만, 챕터 5에서는 배달 완료를 기다리기 위해 아무것도 발행하지 않습니다.

[작성] `OrderOrchestrator.java`

```
@KafkaListener(topics = "delivery-created", groupId = "orchestrator")
public void deliveryCreated(DeliveryCreatedEvent event) {
    int orderId = event.getOrderId();
    WorkflowState state = states.get(orderId);
    if (state == null) return;

    // 실패: 4장과 동일 (재고 복구 → 주문 취소)

    // 성공: 배달 완료를 기다린다 (complete-order-command 발행하지 않음)
    states.remove(orderId); // 배달 완료는 별도 리스너에서 처리하므로 워크플로우 상태 정리
}
```

실패 처리는 챕터 4와 동일합니다. 핵심 변경은 성공 시입니다. 챕터 4에서는 여기서 바로 주문 완료 명령(`complete-order-command`)을 발행했지만, 이제는 아무것도 하지 않고 배달기사의 완료 API 호출을 기다립니다.

5.5.2 deliveryCompleted 추가 : 배달 완료 시 주문 완료

[작성] `handler/OrderOrchestrator.java`

```
@KafkaListener(topics = "delivery-completed", groupId = "orchestrator")
public void deliveryCompleted(DeliveryCompletedEvent event) {
```

```
// 배달기사가 완료 API를 호출한 시점 → 주문 완료 명령 발행
kafkaTemplate.send(
    "complete-order-command",
    String.valueOf(event.getOrderId()),
    new CompleteOrderCommand(event.getOrderId())
);
}
```

배달기사가 `PUT /api/deliveries/{id}/complete` 를 호출하면, `delivery-service`가 배달 완료 (`delivery-completed`) 이벤트를 발행합니다. `orchestrator`가 이를 받아 주문 완료 명령 (`complete-order-command`)을 발행하고, `order-service`가 주문을 완료 처리한 뒤 `WebSocket`으로 사용자에게 알림을 보냅니다.

5.5.3 전체 Kafka 토픽 맵 (최종)

챕터 5에서 배달 완료(`delivery-completed`) 토픽이 추가되어, 최종적으로 10개 토픽을 사용합니다.

토픽	발행	구독	목적
order-created	order-service	orchestrator	새 주문 발생
decrease-product-command	orchestrator	product-service	재고 감소 명령
product-decreased	product-service	orchestrator	재고 감소 결과
create-delivery-command	orchestrator	delivery-service	배달 생성 명령
delivery-created	delivery-service	orchestrator	배달 생성 결과
delivery-completed	delivery-service	orchestrator	배달 완료 이벤트 (챕터 5 신규)
complete-order-command	orchestrator	order-service	주문 완료 명령
cancel-order-command	orchestrator	order-service	주문 취소 명령 (롤백)
increase-product-command	orchestrator	product-service	재고 복구 명령 (롤백)
cancel-delivery-command	orchestrator	delivery-service	배달 취소 명령 (롤백)

5.6 order-service : STOMP로 실시간 Push 구현

마지막 퍼즐입니다. order-service가 주문 완료 명령(`complete-order-command`)을 받아 주문을 완료 처리하고, 동시에 WebSocket으로 사용자에게 알림을 보냅니다.

5.6.1 패키지 구조

order-service/

├─ build.gradle

├─ src/main/java/.../

└─ core/

← [참고] WebSocket 의존성 추가

```

| | └─ config/
| |   └─ WebSocketConfig.java ← [작성] STOMP WebSocket 설정
| |   └─ filter/
| |       └─ JwtAuthenticationFilter.java ← [작성] WebSocket 경로 필터 제외
└─ usecase/
    └─ OrderService.java ← [작성] completeOrder에 WebSocket Push 추가

```

5.6.2 의존성 추가

`build.gradle` 에 `websocket`이 추가됩니다.

[참고] `build.gradle` (order-service)

```
implementation 'org.springframework.boot:spring-boot-starter-websocket'
```

5.6.3 WebSocket 설정

`WebSocketConfig`는 `WebSocket` 기능을 활성화하고, 클라이언트가 `/api/ws/orders` 로 실시간 연결할 수 있도록 엔드포인트를 등록합니다. 클라이언트가 이 엔드포인트로 연결한 뒤 `/topic/orders/{userId}` 채널을 구독하면, 서버가 해당 채널로 보낸 메시지를 실시간으로 수신할 수 있습니다.

[작성] `core/config/WebSocketConfig.java`

```

@Configuration
@EnableWebSocketMessageBroker // STOMP WebSocket 브로커 활성화
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        // /topic 접두사로 메시지 라우팅 (Kafka 토픽과는 다른 개념)
        config.enableSimpleBroker("/topic");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // WebSocket 연결 엔드포인트 - withSockJS()는 미지원 브라우저용 폴백
        registry.addEndpoint("/api/ws/orders").setAllowedOriginPatterns("*").withSockJS();
    }
}

```

5.6.4 JwtAuthenticationFilter 수정

`/api/ws` 경로를 JWT 필터에서 제외해야 합니다. 제외하지 않으면 WebSocket 연결이 401로 실패합니다. 토큰은 WebSocket 연결 시 쿼리 파라미터(`?token=`)로 별도 전달되므로, 필터에서는 이 경로를 건너뛰어야 합니다.

[작성] `core/filter/JwtAuthenticationFilter.java` 의 `shouldNotFilter` 에 `/api/ws` 경로를 추가합니다.

```
@Override
protected boolean shouldNotFilter(HttpServletRequest request) {
    String path = request.getRequestURI();
    return path.equals("/login") ||
        path.startsWith("/h2-console") ||
        path.startsWith("/api/ws"); // 추가: WebSocket 경로 제외
}
```

5.6.5 SimpMessagingTemplate : 주문 완료 시 Push 발송

`completeOrder` 메서드에서 `/topic/orders/{userId}` 채널에 메시지를 보내면 `userId`에 해당하는 사용자만 알림을 수신합니다.

SimpMessagingTemplate: Spring이 제공하는 메시지 전송 도구입니다.
`convertAndSend(destination, payload)` 메서드로 지정한 채널을 구독한 모든 클라이언트에게 메시지를 Push합니다.

아래 메서드를 구현합니다.

[작성] `usecase/OrderService.java` (`completeOrder` 메서드 수정)

```
private final SimpMessagingTemplate messagingTemplate; // 추가: 생성자 주입

@Transactional
public void completeOrder(int orderId) {
    Order findOrder = orderRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    findOrder.complete();
    messagingTemplate.convertAndSend("/topic/orders/" + findOrder.getUserId(), (Object) Map.of("orderId",
        orderId)); // 추가: WebSocket Push
}
```

서버 측 WebSocket 구현이 완료되었습니다. 이제 이 클라이언트를 Kubernetes에서 서빙하기 위한 프론트엔드 인프라를 구성합니다.

5.6.6 프론트엔드 : Nginx와 SockJS 클라이언트

WebSocket 실시간 알림을 테스트하기 위한 프론트엔드 클라이언트를 구성합니다. Nginx로 정적 HTML을 서빙하면서, API 요청은 gateway-service로 프록시합니다.

```
frontend/
├── index.html # [참고] WebSocket 테스트 HTML
└── nginx.conf # [참고] 정적 파일 서빙 + WebSocket 프록시

gateway/
└── nginx.conf # [참고] WebSocket 경로 추가
```

index.html : WebSocket 테스트 클라이언트

프론트엔드는 간단한 HTML 페이지로, JWT 토큰을 입력받아 WebSocket 구독을 설정하고 주문 완료 알림을 실시간으로 표시합니다. 핵심인 WebSocket 연결 부분만 보면 다음과 같습니다.

전체 코드는 GitHub에서 확인하세요.

[참고] [frontend/index.html](#) (WebSocket 연결 부분)

```
// SockJS로 WebSocket 연결 생성
stomp = Stomp.over(new SockJS('/api/ws/orders?token=' + TOKEN));

// STOMP 연결 후 채널 구독
stomp.connect({}, function () {
  stomp.subscribe('/topic/orders/' + userId, function (msg) {
    const data = JSON.parse(msg.body);
    status.textContent = '주문 완료! (주문번호: ' + data.orderId + ')';
  });
});
```

[SockJS](#) 로 WebSocket 연결을 만들고, [STOMP](#) 로 `/topic/orders/{userId}` 채널을 구독합니다. 서버가 이 채널로 메시지를 보내면 콜백이 실행되어 화면에 주문 완료를 표시합니다.

nginx.conf : WebSocket 프록시 설정

Nginx가 중간에 있으면 WebSocket 연결이 일반 HTTP로 처리되어 끊어집니다. `/api/ws/` 경로에 **Upgrade** 헤더를 설정하여 “이 연결은 WebSocket이니 끊지 말라”고 알려줘야 합니다.

Nginx WebSocket 프록시(Upgrade 헤더): Nginx가 WebSocket 연결을 프록시할 때 필요한 설정입니다. nginx 설정에 `upgrade 헤더` 를 설정하여 HTTP 연결을 WebSocket 프로토콜로 전환(upgrade)하도록 백엔드에 전달합니다.

정적 파일을 제공하면서 `/login` , `/api/` 요청은 gateway-service로 전달합니다. `/api/ws/` 블록이 핵심입니다.

[참고] [frontend/nginx.conf](#)

```
# /api/ws/ 경로로 들어오는 WebSocket 요청을 gateway로 전달
location /api/ws/ {
    proxy_pass http://gateway;      # gateway-service로 요청 전달
    proxy_http_version 1.1;        # HTTP/1.1 사용 (WebSocket 필수)
    proxy_set_header Upgrade $http_upgrade; # 클라이언트의 Upgrade 헤더 전달
    proxy_set_header Connection "upgrade"; # 연결을 WebSocket으로 전환
}
```

gateway-service의 nginx 설정도 동일하게 `upgrade 헤더` 를 추가합니다.

[참고] [gateway/nginx.conf](#) (WebSocket 추가분)

```
# 기존 location 블록들에 추가
location /api/ws/ {
    proxy_pass http://order-service;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

5.7 전체 시스템 통합 테스트

모든 구현이 완료되었습니다. 이제 전체 시스템을 실행하고 처음부터 끝까지 한 번에 흐름을 확인합니다.

5.7.1 K8s 매니페스트

챕터 4 대비 frontend 서비스가 새로 추가됩니다. `k8s/frontend/` 폴더에 Deployment, Service, Ingress가 정의되어 있습니다.

파일	역할
<code>frontend-deploy.yml</code>	Nginx 기반 프론트엔드 Pod
<code>frontend-service.yml</code>	클러스터 내부 접근용 Service
<code>frontend-ingress.yml</code>	외부 요청을 frontend-service로 라우팅

이번 챕터부터는 Ingress가 gateway-service가 아닌 **frontend-service**를 가리킵니다. 프론트엔드의 Nginx가 정적 파일을 직접 제공하고, `/api/` 요청만 gateway-service로 전달합니다.

5.7.2 이미지 빌드

```
minikube image build -t metacoding/db:3 ./db
minikube image build -t metacoding/gateway:3 ./gateway
minikube image build -t metacoding/order:3 ./order
minikube image build -t metacoding/product:3 ./product
minikube image build -t metacoding/user:3 ./user
minikube image build -t metacoding/delivery:3 ./delivery
minikube image build -t metacoding/orchestrator:3 ./orchestrator
minikube image build -t metacoding/frontend:3 ./frontend
```

5.7.3 배포

Kafka를 먼저 배포하고 ready 상태를 확인한 다음 나머지를 배포합니다.

```
# 1. 네임스페이스 생성 (최초 1회)
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/gateway
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
```



```
kubectl apply -f k8s/delivery
kubectl apply -f k8s/orchestrator
kubectl apply -f k8s/frontend
```

```
# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```

모든 Pod가 Running 상태가 될 때까지 대기합니다.

```
kubectl get pods -n metacoding
```

실행결과

```
$ kubectl get pods -n metacoding
```

NAME	READY	STATUS	RESTARTS	AGE
db-deploy-5c94cbfb5f-rr6nk	1/1	Running	0	3m7s
delivery-deploy-56cfc89bdd-rsq4	1/1	Running	1 (63s ago)	3m3s
frontend-deploy-66b885b47-g9mpw	1/1	Running	0	3m1s
gateway-deploy-5d4c49c94c-snm6w	1/1	Running	1 (2m58s ago)	3m6s
kafka-deploy-69fc99c86-fwpt8	1/1	Running	0	3m9s
orchestrator-deploy-65bdd754c6-kbb5c	1/1	Running	0	3m2s
order-deploy-665d89c8fc-vjw9k	1/1	Running	1 (73s ago)	3m6s
product-deploy-6f5745c6b9-hwcbr	1/1	Running	1 (71s ago)	3m5s
user-deploy-5d786978bd-bgbkz	1/1	Running	1 (69s ago)	3m4s

그림 0-6: Pod 상태 확인

5.7.4 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel` 을 실행합니다.

```
minikube tunnel
```

터널이 실행되면 <http://127.0.0.1:80> 로 프론트엔드에 접속할 수 있습니다.

5.7.5 통합 테스트 시나리오

Step 1: WebSocket 연결 및 주문 생성 (클라이언트 역할)

브라우저를 통해 index.html에 접속합니다.

브라우저 <http://127.0.0.1:80/index.html>



그림 0-7: 브라우저에서 index.html 접속 화면

WebSocket 연결을 위해 토큰을 입력합니다. 여기서 **Bearer** 접두사를 제외한 토큰 값만 입력합니다. 토큰을 입력하고 주문하기 버튼을 클릭합니다. index.html이 내부적으로 WebSocket에 연결하고 `/topic/orders/{userId}` 채널을 구독한 뒤 주문 요청을 보냅니다.



그림 0-8: 토큰 입력 후 주문하기 버튼 클릭

브라우저 **F12** - **Console** 에서 WebSocket이 연결됨을 확인할 수 있습니다.

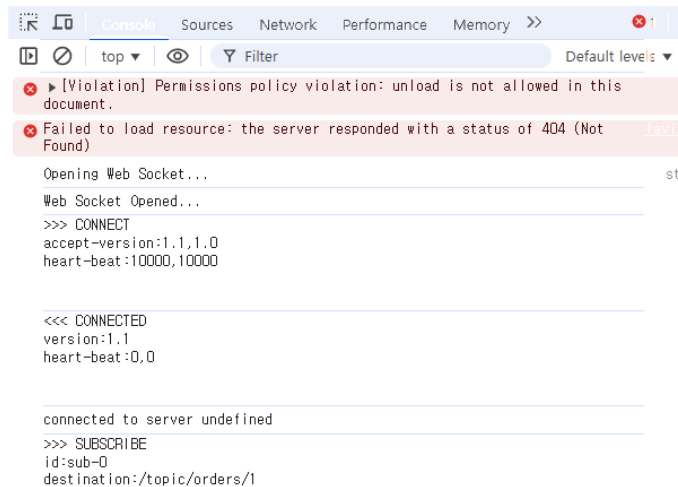


그림 0-9: 브라우저 Console에서 WebSocket 연결 확인

Hoppscotch로 생성된 주문을 확인하면 **PENDING** 상태로 머물러 있습니다.

GET <http://127.0.0.1:80/api/orders/4>

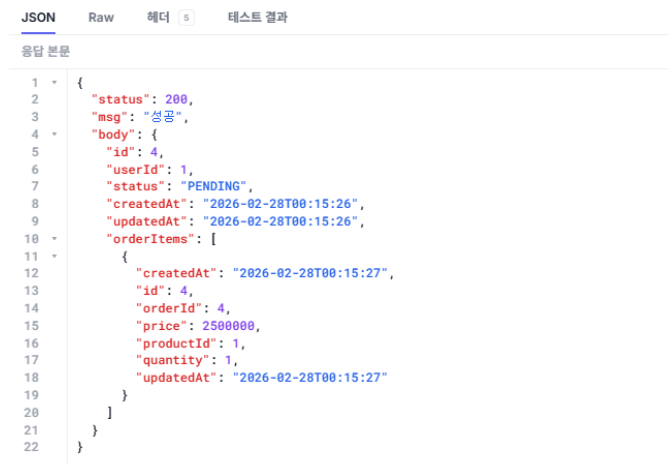


그림 0-10: 주문 조회 결과 — PENDING 상태

Step 2: 배달 완료 (배달 기사 역할)

먼저 생성된 배달을 확인해보겠습니다.

GET <http://127.0.0.1:80/api/deliveries/4>

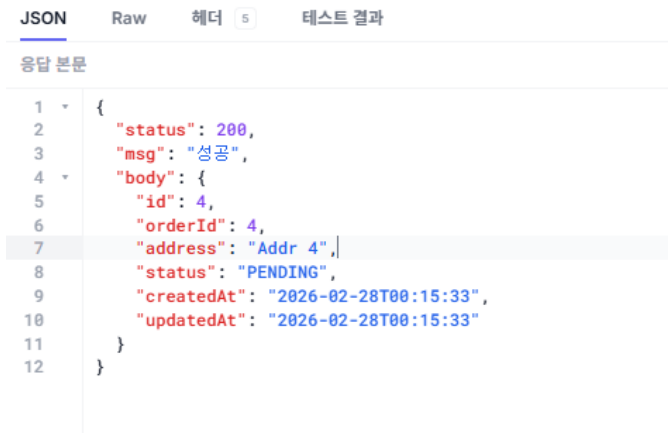


그림 0-11: 배달 조회 결과 - PENDING 상태

배달 ID가 4인 배달이 **PENDING** 상태로 생성되었습니다.

배달 기사가 물건을 전달한 뒤 배달 완료 처리를 합니다.

PUT `http://127.0.0.1:80/api/deliveries/4/complete`

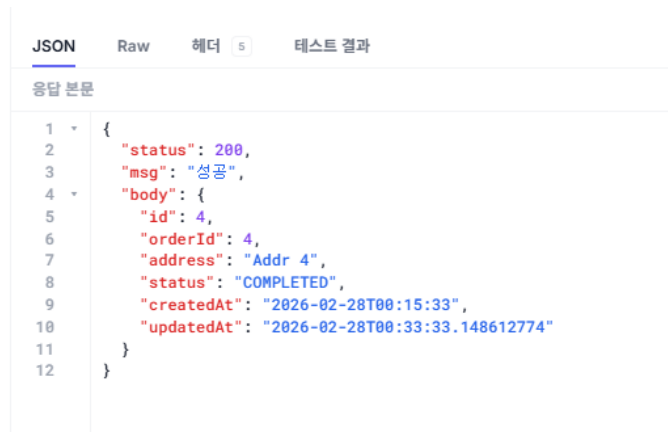


그림 0-12: 배달 완료 API 호출 결과 - COMPLETED 상태

Step 3: 주문 완료 및 웹소켓 응답 확인

배달 완료 처리 후, 주문 완료 명령에 의해 주문이 최종적으로 **COMPLETED** 상태가 되었습니다.

GET `http://127.0.0.1:80/api/orders/4`

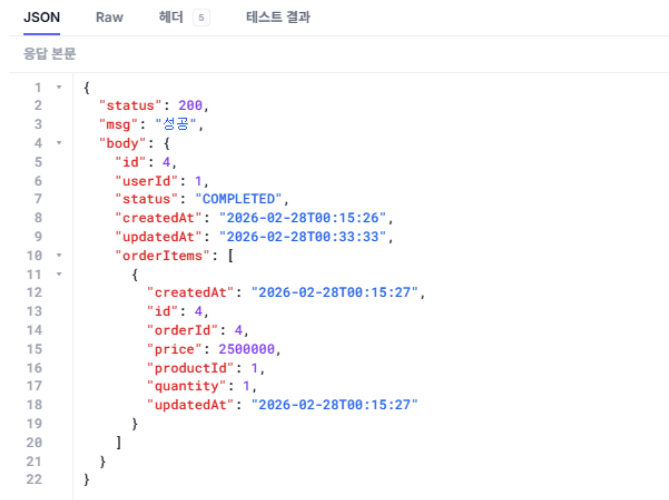


그림 0-13: 주문 조회 결과 — COMPLETED 상태

주문이 완료되면 WebSocket이 클라이언트에게 주문 완료 메시지를 전송합니다. WebSocket 응답을 수신하면 클라이언트 화면이 주문 완료 상태로 변경됩니다.



그림 0-14: WebSocket 알림 수신 — 클라이언트 화면에 주문 완료 표시

브라우저의 Console 창에서 WebSocket이 응답한 메시지 로그를 확인할 수 있습니다.

```
<<< MESSAGE
destination:/topic/orders/1
content-type:application/json
subscription:sub-0
message-id:ydglhrfy-0
content-length:13
```

```
{ "orderId":1 }
```

```
<<< MESSAGE
destination:/topic/orders/1
content-type:application/json
subscription:sub-0
message-id:ydglhrfy-1
content-length:13
```

```
{ "orderId":4 }
```

`ctrl` `j` to turn on code suggestions. Don't show again

그림 0-15: 브라우저 Console에서 WebSocket 응답 메시지 로그 확인

5.7.6 전체 흐름 요약

[최종 완성 시스템 전체 흐름]

1. **POST /api/orders** → **order-service (PENDING 저장)**

└─ **Kafka: order-created**

2. **orchestrator: order-created** 수신

└─ **Kafka: decrease-product-command**

3. **product-service: 재고 감소**

└─ **Kafka: product-decreased { success: true }**

4. **orchestrator: 재고 감소 확인**

└─ **Kafka: create-delivery-command**

5. **delivery-service: 배달 생성 (PENDING 상태)**

└─ **Kafka: delivery-created { success: true }**

6. **orchestrator: delivery-created** 수신 → 대기

7. **PUT /api/deliveries/{id}/complete** (배달 기사)

delivery-service: 배달 COMPLETED

└─ **Kafka: delivery-completed**

8. **orchestrator: delivery-completed** 수신

└ Kafka: **complete-order-command**

9. **order-service: 주문 COMPLETED**

└ WebSocket Push: /topic/orders/{userId}

10. 클라이언트: 실시간 알림 수신 ✓

정리

이번 챕터에서 완성한 것을 정리합니다.

- 배달 완료 라이프사이클: 배달 생성 시 **PENDING**, 배달 기사 API 호출 시 **COMPLETED** 로 실제 배달 완료 시점을 정확히 반영합니다.
- **delivery-completed** 토픽 추가: delivery-service가 완료 이벤트를 발행하고 orchestrator가 처리합니다. 이로써 챕터 5에서 실제 사용하는 Kafka 토픽은 10개가 됩니다.
- **orchestrator** 변경: 배달 생성(**delivery-created**) 성공 후 대기, 배달 완료(**delivery-completed**) 수신 후 주문 완료 처리. 실패 시에는 재고를 복구합니다.
- **WebSocket Push**: order-service가 **SimpMessagingTemplate** 으로 사용자별 채널에 실시간 알림을 보냅니다.

마치며

■ 챕터별 비교

	2장	3장	4장	5장
통신	REST 동기	REST 동기	Kafka 비동기	Kafka 비동기
트랜잭션	보상 트랜잭션	보상 트랜잭션	Orchestration Saga	Orchestration Saga
아키텍처	레이어드	Clean (UseCase)	Clean + 이벤트	Clean + 이벤트
배포	로컬	Kubernetes	K8s + Kafka	K8s + Kafka
배달 완료	자동 (생성 시)	자동 (생성 시)	자동 (생성 시)	수동 API 호출
실시간 알림	없음	없음	없음	WebSocket Push

■ 마지막으로

이 책은 MSA의 완성된 정답을 보여주려 하지 않았습니다. 대신, 동기 통신에서 시작하여 왜 비동기가 필요한지, 왜 분산 트랜잭션이 어려운지를 직접 코드로 경험하게 하려 했습니다.

2장에서 보상 트랜잭션을 try-catch로 직접 구현하며 “이것이 복잡하다”는 것을 느꼈을 것입니다.

3장에서 Clean Architecture로 구조를 개선하고, Gateway로 단일 진입점을 만들어 Kubernetes에 배포하며 “이것이 운영이다”를 경험했습니다. 4장에서 Kafka로 서비스를 분리하며 “비동기가 왜 필요한가”를 이해했습니다. 5장에서 WebSocket Push로 비동기 처리 결과를 실시간으로 전달하며 “분리된 서비스들이 하나의 흐름으로 연결되는 순간”을 경험했습니다.

처음 만나는 MSA는 낯설고 복잡하게 느껴집니다. 하지만 하나의 주문이 5개 서비스를 거쳐 처리되고, 그 결과가 실시간으로 알림으로 오는 순간, MSA가 왜 이렇게 설계됐는지 몸으로 이해하게 됩니다.

이 책에서 만든 시스템이 여러분의 다음 프로젝트를 설계할 때 든든한 참고가 되길 바랍니다.